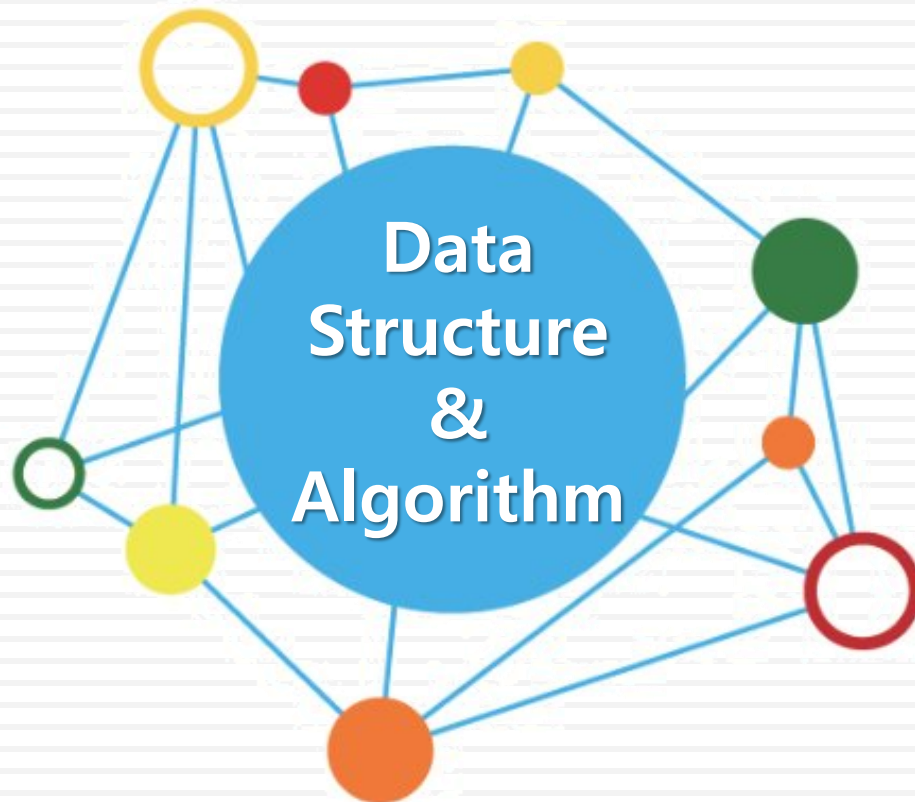


데이터 구조론



유길현

정렬

❖ 정렬(Sort)이란 ?

- 정렬은 물건을 크기 순으로 오름차순이나 내림차순으로 나열하는 것
- 컴퓨터 공학분야에서 기본적이고 중요한 알고리즘의 하나
- 자료 탐색에 있어서 필수

❖ 정렬 방식의 분류

- 내부 정렬(Internal Sort)
 - 정렬할 자료를 메인 메모리에 올려서 정렬하는 방식
 - 정렬 속도가 빠르지만 정렬할 수 있는 자료의 양이 메인 메모리의 용량에 따라 제한됨
- 외부 정렬(External Sort)
 - 정렬할 자료를 보조 기억장치에서 정렬하는 방식
 - 대용량의 보조 기억 장치를 사용하기 때문에 내부 정렬보다 속도는 떨어지지만 내부 정렬로 처리할 수 없는 대용량의 자료에 대한 정렬 가능

❖ 정렬 알고리즘의 선택

- 모든 경우에 최적의 정렬 알고리즘은 없기 때문에 각 응용분야에 적합한 정렬 방법을 사용해야 함
 - 레코드 수의 많고 적음
 - 레코드 크기의 크고 작음
 - Key의 특성(문자, 정수, 실수 등)
 - 메모리 내부/외부 정렬
- 정렬 알고리즘의 평가기준
 - 비교 횟수의 많고 적음
 - 이동 횟수의 많고 적음

❖ 정렬 알고리즘의 선택

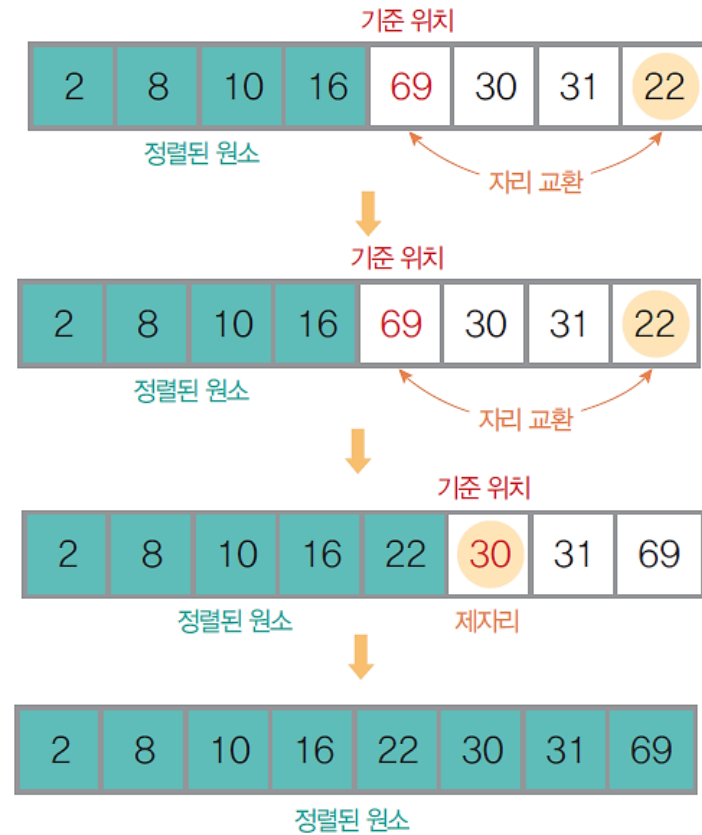
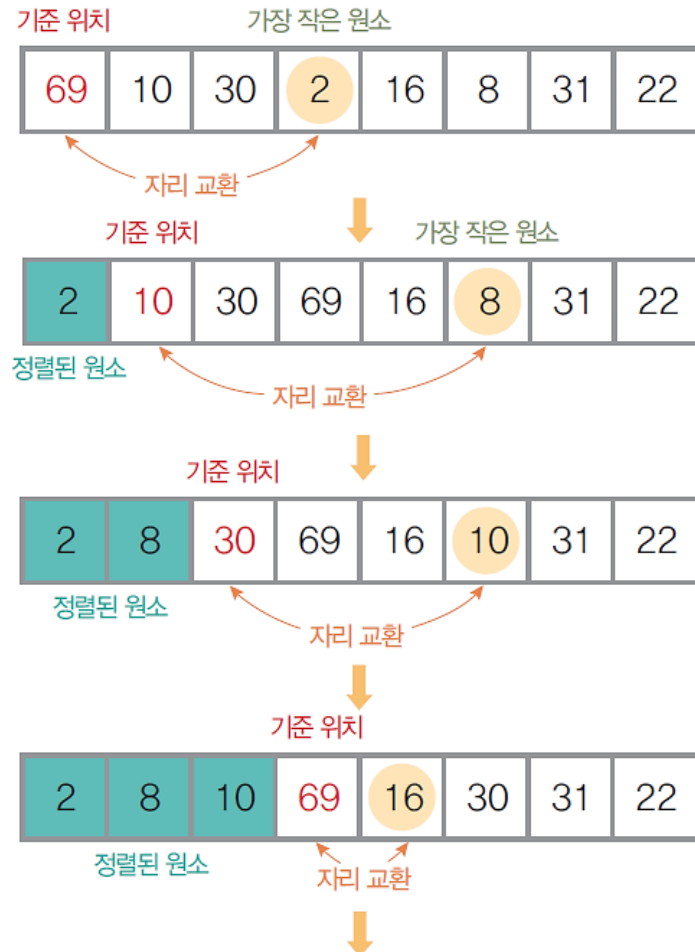
- 단순하지만 비 효율적인 방법
 - 삽입정렬, 선택정렬, 버블정렬
- 복잡하지만 효율적인 방법
 - 퀵정렬, 힙정렬, 합병정렬, 기수정렬

❖ 선택 정렬(Selected Sort)

- 전체 원소들 중에서 기준 위치에 맞는 원소를 선택하여 자리를 교환하는 방식으로 정렬
- 수행방법
 - 전체 원소 중 가장 작은 원소를 찾아 첫 번째 원소와 자리를 교환
 - 두 번째로 작은 원소를 찾아 선택하여 두 번째 원소와 자리를 교환
 - 세 번째로 작은 원소를 찾아 선택하여 세 번째 원소와 자리를 교환
 - 이 과정을 반복하면서 정렬을 완성

❖ 선택 정렬(Selected sort)

- 정렬되지 않은 {69, 10, 30, 2, 16, 8, 31, 22}의 자료를 선택 정렬 방법으로 정렬하는 과정



❖ 선택 정렬 알고리즘

- 크기가 n 인 배열 $a[]$ 의 원소를 선택 정렬하는 알고리즘

알고리즘	선택 정렬
<pre>SelectionSort($a[]$, n) for ($i \leftarrow 0$; $i < n$; $i \leftarrow i + 1$) do { $a[i]$, ..., $a[n-1]$ 중에서 가장 작은 원소 $a[k]$를 선택해 $a[i]$와 교환한다.; } end SelectionSort()</pre>	

❖ 선택 정렬 알고리즘

- 메모리 사용공간
 - n개의 원소에 대한 n개의 메모리 사용
- 비교횟수
 - 1단계 : 첫 번째 원소를 기준으로 n개의 원소 비교
 - 2단계 : 두 번째 원소를 기준으로 마지막 원소까지 n-1개의 원소 비교
 - 3단계 : 세 번째 원소를 기준으로 마지막 원소까지 n-2개의 원소 비교
 - i 단계 : i 번째 원소를 기준으로 n-i개의 원소 비교

$$\text{전체 비교횟수} : (n-1) + (n-2) + \dots + 2 + 1 = \sum_{i=1}^{n-1} n - i = \frac{n(n-1)}{2}$$

- 어떤 경우에서나 비교횟수가 같으므로 시간 복잡도는 $O(n^2)$ 이 됨

❖ 선택 정렬하기 프로그램

```
#define _CRT_SECURE_NO_WARNINGS
#include "stdio.h"

typedef int element;
int size;

// 선택 정렬하는 연산
void SelectionSort(int a[], int size)
{
    int i, j, t, min;
    element temp;
    printf("\n정렬할 원소 : ");
    for (t = 0; t < size; t++)
        printf("%d ", a[t]); // 정렬 전의 원소 출력
    printf("\n\n<<<<<<<<< 선택 정렬 수행 >>>>>>>>>\n");
```

❖ 선택 정렬하기 프로그램

```
for (i = 0; i < size - 1; i++)
{
    min = i;
    for (j = i + 1; j < size; j++)
    {
        if (a[j] < a[min])
            min = j;
    }
    temp = a[i];
    a[i] = a[min];
    a[min] = temp;
    printf("Wn%d단계 : ", i + 1);
    for (t = 0; t < size; t++)
        printf("%3d ", a[t]);
}
}
```

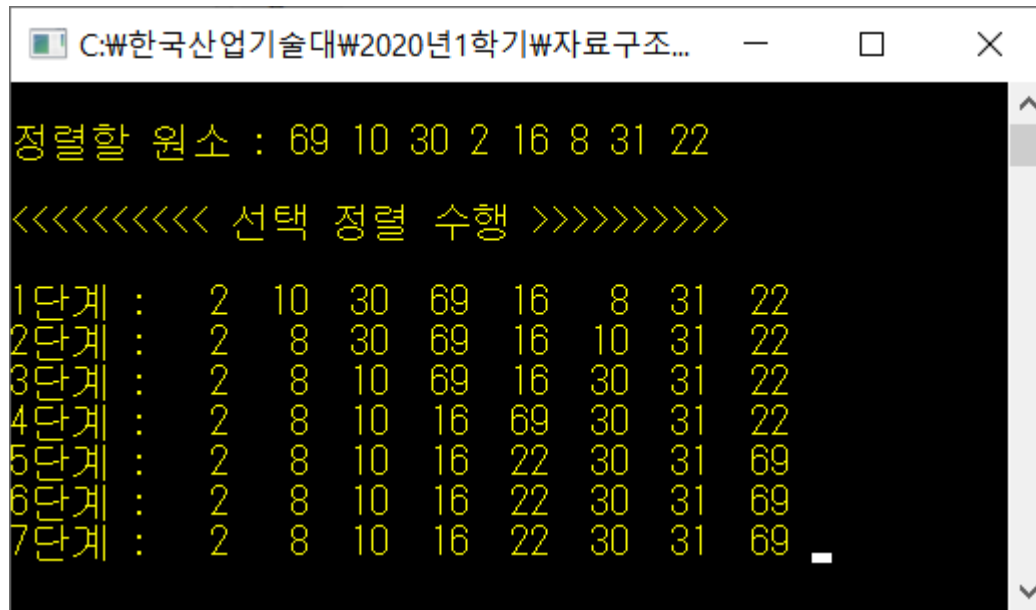
❖ 선택 정렬하기 프로그램

```
void main()
{
    // 정렬할 초기 원소 배열
    element list[8] = { 69, 10, 30, 2, 16, 8, 31, 22 };
    size = 8;
    SelectionSort(list, size);    // 선택 정렬 연산 호출

    getchar();
}
```

❖ 선택 정렬하기 프로그램 : [교재 500p](#)

■ 실행 결과



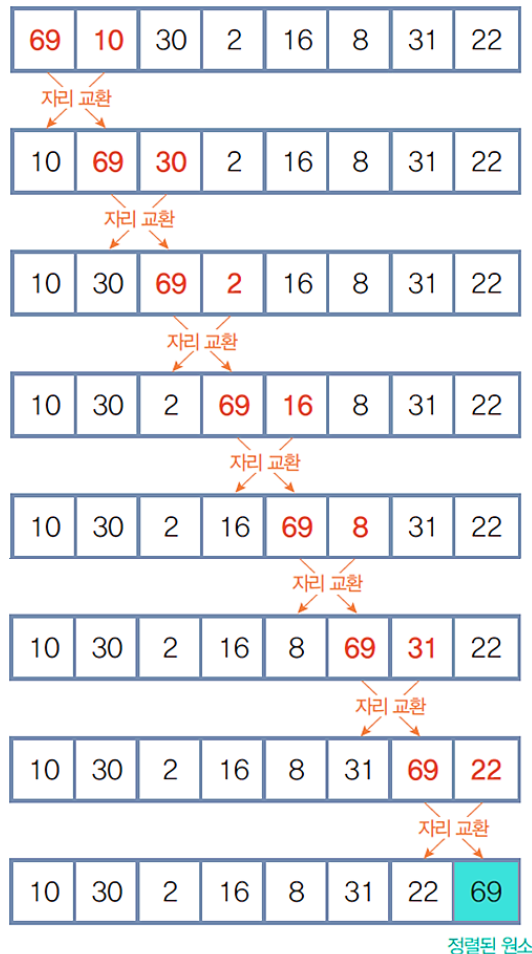
```
C:\₩한국산업기술대₩2020년1학기₩자료구조...
정렬할 원소 : 69 10 30 2 16 8 31 22
<<<<<<<<< 선택 정렬 수행 >>>>>>>>>
1단계 : 2 10 30 69 16 8 31 22
2단계 : 2 8 30 69 16 10 31 22
3단계 : 2 8 10 69 16 30 31 22
4단계 : 2 8 10 16 69 30 31 22
5단계 : 2 8 10 16 22 30 31 69
6단계 : 2 8 10 16 22 30 31 69
7단계 : 2 8 10 16 22 30 31 69
```

❖ 버블 정렬(Bubbles Sort)

- 인접한 두 개의 원소를 비교하여 자리를 교환하는 방식
 - 첫 번째 원소부터 마지막 원소까지 반복하여 한 단계가 끝나면 가장 큰 원소가 마지막 자리로 정렬
 - 첫 번째 원소부터 인접한 원소끼리 계속 자리를 교환하면서 맨 마지막 자리로 이동하는 모습이 물 속에서 물 위로 올라오는 물방울 모양과 같다고 하여 버블(bubble) 정렬이라 함.

❖ 버블 정렬(Bubbles Sort)

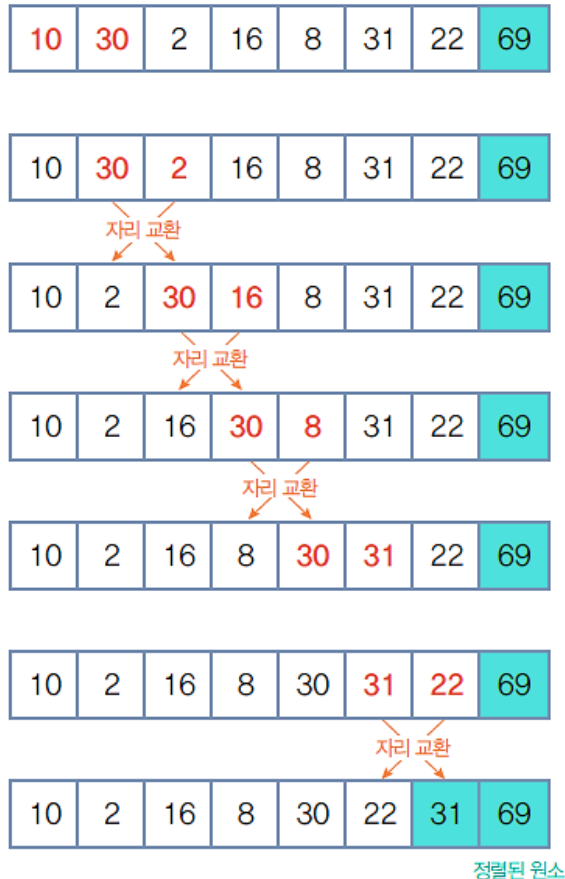
- 정렬되지 않은 {69, 10, 30, 2, 16, 8, 31, 22}의 자료들을 버블 정렬 방법으로 정렬하는 과정



- ① 1단계 : 인접한 두 원소를 비교하여 자리를 교환하는 작업을 첫 번째 원소 부터 마지막 원소까지 차례로 반복하여 가장 큰 원소 69를 마지막 자리로 정렬.

❖ 버블 정렬(Bubbles Sort)

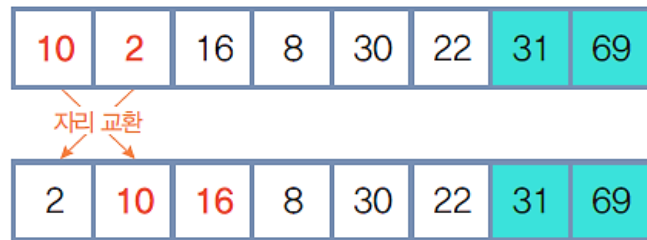
- 정렬되지 않은 {69, 10, 30, 2, 16, 8, 31, 22}의 자료들을 버블 정렬 방법으로 정렬하는 과정



- ② 2단계 : 정렬하지 않은 원소에 대해 두 번째 버블 정렬을 수행하면, 나머지 원소 중에서 가장 큰 원소 31이 끝에서 둘째 자리로 정렬

❖ 버블 정렬(Bubbles Sort)

- 정렬되지 않은 {69, 10, 30, 2, 16, 8, 31, 22}의 자료들을 버블 정렬 방법으로 정렬하는 과정



정렬된 원소

- ③ 3단계 : 세 번째 버블 정렬을 수행하면, 나머지 원소 중에서 가장 큰 원소 30이 끝에서 셋째 자리로 정렬

❖ 버블 정렬(Bubbles Sort)

- 정렬되지 않은 {69, 10, 30, 2, 16, 8, 31, 22}의 자료들을 버블 정렬 방법으로 정렬하는 과정

2	10	8	16	22	30	31	69
---	----	---	----	----	----	----	----

2	10	8	16	22	30	31	69
---	----	---	----	----	----	----	----

자리 교환

2	8	10	16	22	30	31	69
---	---	----	----	----	----	----	----

2	8	10	16	22	30	31	69
---	---	----	----	----	----	----	----

2	8	10	16	22	30	31	69
---	---	----	----	----	----	----	----

정렬된 원소

- ④ 4단계 : 번째 버블 정렬을 정렬하지 않은 원소에 대해 수행하면, 나머지 원소 중에서 가장 큰 원소 22가 끝에서 넷째 자리로 정렬

❖ 버블 정렬(Bubbles Sort)

- 정렬되지 않은 {69, 10, 30, 2, 16, 8, 31, 22}의 자료들을 버블 정렬 방법으로 정렬하는 과정

⑤ 5단계 : 다섯 번째 버블 정렬을 수행하면 나머지 원소 중에서 가장 큰 원소 16이 끝에서 다섯째 자리로 정렬

2	8	10	16	22	30	31	69
---	---	----	----	----	----	----	----

2	8	10	16	22	30	31	69
---	---	----	----	----	----	----	----

2	8	10	16	22	30	31	69
---	---	----	----	----	----	----	----

2	8	10	16	22	30	31	69
---	---	----	----	----	----	----	----

정렬된 원소

❖ 버블 정렬(Bubbles Sort)

- 정렬되지 않은 {69, 10, 30, 2, 16, 8, 31, 22}의 자료들을 버블 정렬 방법으로 정렬하는 과정

⑥ 6단계 : 여섯 번째 버블 정렬을 수행하면 나머지 원소 중에서 가장 큰 원소 10이 끝에서 여섯째 자리로 정렬

2	8	10	16	22	30	31	69
---	---	----	----	----	----	----	----

2	8	10	16	22	30	31	69
---	---	----	----	----	----	----	----

2	8	10	16	22	30	31	69
---	---	----	----	----	----	----	----

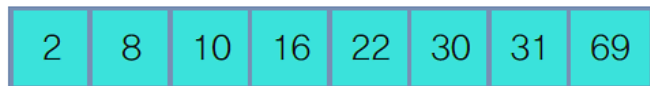
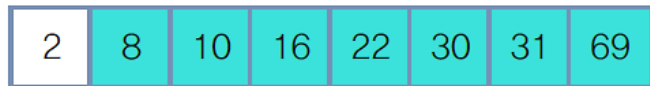
정렬된 원소

❖ 버블 정렬(Bubbles Sort)

- 정렬되지 않은 {69, 10, 30, 2, 16, 8, 31, 22}의 자료들을 버블 정렬 방법으로 정렬하는 과정



정렬된 원소



정렬된 원소

⑦ 7단계 : 나머지 원소 중에서 가장 큰 원소인 8이 끝에서 일곱째 자리로 정렬

⑧ 마지막에 남은 첫째 원소는 전체 원소 중에서 가장 작은 원소로, 가장 앞자리에 남아 이미 정렬된 상태가 됨으로 실행을 종료

❖ 버블 정렬 알고리즘

- 크기가 n 인 배열 $a[]$ 의 원소를 버블 정렬하는 알고리즘

알고리즘	버블 정렬
<pre>bubbleSort($a[]$, n) for ($i \leftarrow n-1$; $i \geq 0$; $i \leftarrow i-1$) do { for($j \leftarrow 0$; $j < i$; $j \leftarrow j+1$) do { if($a[j] > a[j+1]$) then { temp $\leftarrow$ $a[j]$; $a[j] \leftarrow a[j+1]$; $a[j+1] \leftarrow$ temp } } } end bubbleSort()</pre>	

❖ 메모리 사용공간

- n개의 원소에 대하여 n개의 메모리 사용

❖ 연산 시간

- 최선의 경우 : 자료가 이미 정렬되어있는 경우
 - 비교횟수 : i번째 원소를 (n-i)번 비교하므로, $\frac{n(n-1)}{2}$ 번
 - 자리교환횟수 : 자리교환이 발생하지 않음
- 최악의 경우 : 자료가 역순으로 정렬되어있는 경우
 - 비교횟수 : i번째 원소를 (n-i)번 비교하므로, $\frac{n(n-1)}{2}$ 번
 - 자리교환횟수 : i번째 원소를 (n-i)번 교환하므로, $\frac{n(n-1)}{2}$ 번
- 최선의 경우와 최악의 경우에 대한 평균 시간 복잡도를 빅-오 (Big Oh) 표기법으로 나타내면 $O(n^2)$ 이 됨

❖ 연산 시간

▪ 순차 정렬된 경우와 역순 정렬된 경우의 버블 정렬 비교



- 전체 비교 횟수 : $1+2+3+\dots+(n-1) = \frac{n(n-1)}{2}$
- 전체 자리 교환 횟수 : $1+2+3+\dots+(n-1) = \frac{n(n-1)}{2}$
- 시간 복잡도 : $O(n^2)$



- 전체 비교 횟수 : $1+2+3+\dots+(n-1) = \frac{n(n-1)}{2}$
- 전체 자리 교환 횟수 : 0
- 시간 복잡도 : $O(n^2)$

❖ 버블 정렬하기 프로그램

```
#define _CRT_SECURE_NO_WARNINGS
#include "stdio.h"
typedef int element;
int size;

// 버블 정렬하는 연산
void bubbleSort(element a[], int size)
{
    int i, j, t;
    element temp;
    printf("\n정렬할 원소 : ");
    for (t = 0; t < size; t++)
        printf("%d ", a[t]);
```


❖ 버블 정렬하기 프로그램

```
printf("\n<<<<<<<< 버블 정렬 수행 >>>>>>>>");
for (i = size - 1; i > 0; i--)
{
    printf("\n %d단계>>", size - i);
    for (j = 0; j < i; j++)
    {
        if (a[j] > a[j + 1])
        {
            temp = a[j];
            a[j] = a[j + 1];
            a[j + 1] = temp;
        }
        printf("\n\t\t");
        for (t = 0; t < size; t++)
            printf("%3d ", a[t]);
    }
}
```

❖ 버블 정렬하기 프로그램

```
void main()
{
    element list[8] = { 69, 10, 30, 2, 16, 8, 31, 22 };
    size = 8;
    bubbleSort(list, size);
    getchar();
}
```

❖ 버블 정렬하기 프로그램 : [교재 507p](#)

■ 실행 결과

```
C:\₩한국산업기술대W2020년1학기W...  -  □  X

정렬할 원소 : 69 10 30 2 16 8 31 22
<<<<<<<<< 버블 정렬 수행 >>>>>>>>>

1단계>>
  10 69 30 2 16 8 31 22
  10 30 69 2 16 8 31 22
  10 30 2 69 16 8 31 22
  10 30 2 16 69 8 31 22
  10 30 2 16 8 69 31 22
  10 30 2 16 8 31 69 22
  10 30 2 16 8 31 22 69

2단계>>
  10 30 2 16 8 31 22 69
  10 2 30 16 8 31 22 69
  10 2 16 30 8 31 22 69
  10 2 16 8 30 31 22 69
  10 2 16 8 30 31 22 69
  10 2 16 8 30 31 22 69
  10 2 16 8 30 22 31 69

3단계>>
  2 10 16 8 30 22 31 69
  2 10 16 8 30 22 31 69
  2 10 8 16 30 22 31 69
  2 10 8 16 30 22 31 69
  2 10 8 16 30 22 31 69
  2 10 8 16 22 30 31 69

4단계>>
  2 10 8 16 22 30 31 69
  2 8 10 16 22 30 31 69
  2 8 10 16 22 30 31 69
  2 8 10 16 22 30 31 69
```

```
C:\₩한국산업기술대W2020년1학기W...  -  □  X

5단계>>
  2 8 10 16 22 30 31 69
  2 8 10 16 22 30 31 69
  2 8 10 16 22 30 31 69

6단계>>
  2 8 10 16 22 30 31 69
  2 8 10 16 22 30 31 69

7단계>>
  2 8 10 16 22 30 31 69
```

❖ 삽입 정렬(insert sort)의 이해

- 정렬되어 있는 부분집합에 정렬할 새로운 원소의 위치를 찾아 삽입하는 방법
- 정렬할 자료를 두 개의 부분집합 S와 U로 가정
 - 부분집합 S(Sorted Subset) : 정렬된 앞부분의 원소들
 - 부분집합 U(Unsorted Subset) : 아직 정렬되지 않은 나머지 원소들
 - 정렬되지 않은 부분집합 U의 원소를 하나씩 꺼내서 이미 정렬되어있는 부분집합 S의 마지막 원소부터 비교하면서 위치를 찾아 삽입
 - 삽입 정렬을 반복하면서 부분집합 S의 원소는 하나씩 늘리고 부분집합 U의 원소는 하나씩 감소하게 함. 부분집합 U가 공집합이 되면 삽입 정렬이 완성

❖ 삽입 정렬 수행 과정

- 정렬하지 않은 {69, 10, 30, 2, 16, 8, 31, 22} 자료를 삽입 정렬 방법으로 정렬하는 과정
 - 초기 상태 : 첫째 원소는 정렬되어 있는 부분집합 S로, 나머지 원소는 정렬 되지 않은 원소들의 부분집합 U로 가정



$S = \{69\}$

$U = \{10, 30, 2, 16, 8, 31, 22\}$

- ① 1단계 : U의 첫째 원소 10을 S의 마지막 원소 69와 비교하면 $10 < 69$ 이므로 원소 10은 원소 69의 앞자리가 됨. 더 이상 비교할 S의 원소가 없으므로 찾은 위치에 원소 10을 삽입



삽입



$S = \{10, 69\}$

$U = \{30, 2, 16, 8, 31, 22\}$

❖ 삽입 정렬 수행 과정

- ② 2단계 : 현재 U의 첫째 원소 30을 S의 마지막 원소 69와 비교하면 $30 < 69$ 이므로 원소 69의 앞자리 원소 10과 비교. $30 > 10$ 이므로 원소 10과 69 사이에 삽입



삽입

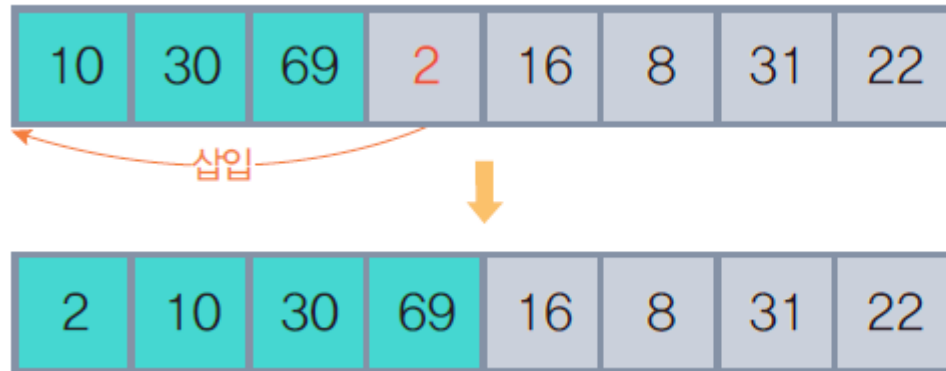


$S = \{10, 30, 69\}$

$U = \{2, 16, 8, 31, 22\}$

❖ 삽입 정렬 수행 과정

- ③ 3단계 : 현재 U의 첫째 원소 2를 S의 마지막 원소 69와 비교하면 $2 < 69$ 이므로 원소 69의 앞자리 원소 30과 비교. $2 < 30$ 이므로 다시 원소 30의 앞자리 원소 10과 비교. $2 < 10$ 이고 더 이상 비교할 S의 원소가 없으므로 원소 10 앞에 삽입



$S = \{2, 10, 30, 69\}$
 $U = \{16, 8, 31, 22\}$

❖ 삽입 정렬 수행 과정

- ④ 4단계 : 현재 U의 첫째 원소 16을 S의 마지막 원소 69와 비교하면 $16 < 69$ 이므로 그 앞자리 원소 30과 비교. $16 < 30$ 이므로 다시 그 앞자리 원소 10과 비교. $16 > 10$ 이므로 원소 10과 30 사이에 삽입

2	10	30	69	16	8	31	22
---	----	----	----	----	---	----	----

삽입

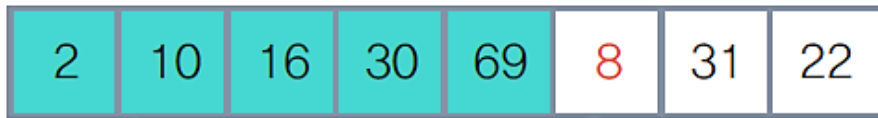
2	10	16	30	69	8	31	22
---	----	----	----	----	---	----	----

$S = \{2, 10, 16, 30, 69\}$

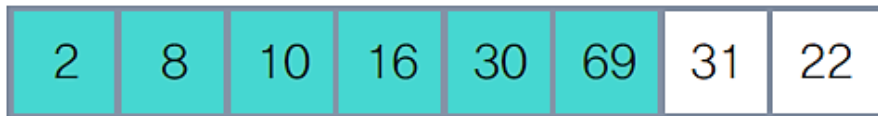
$U = \{8, 31, 22\}$

❖ 삽입 정렬 수행 과정

- ⑤ 5단계 : 현재 U의 첫 번째 원소 8을 S의 마지막 원소 69와 비교하면 $8 < 69$ 이므로 그 앞자리 원소 30과 비교. $8 < 30$ 이므로 그 앞자리 원소 16과 비교. $8 < 16$ 이므로 그 앞자리 원소 10과 비교. $8 < 10$ 이므로 다시 그 앞자리 원소 2와 비교. $8 > 2$ 이므로 원소 2와 10 사이에 삽입



삽입

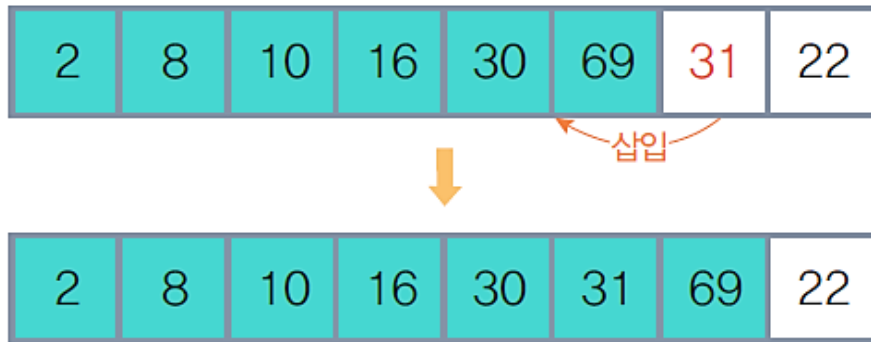


$S = \{2, 8, 10, 16, 30, 69\}$

$U = \{31, 22\}$

❖ 삽입 정렬 수행 과정

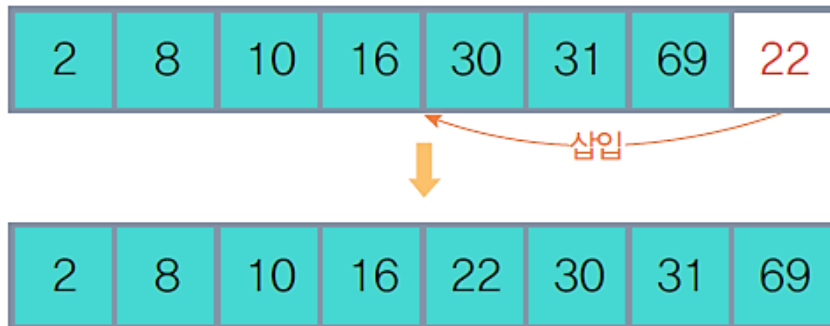
- ⑥ 6단계 : 현재 U의 첫째 원소 31을 S의 마지막 원소 69와 비교하면 $31 < 69$ 이므로 그 앞자리 원소 30과 비교. $31 > 30$ 이므로 원소 30과 69 사이에 삽입



$S = \{2, 8, 10, 16, 30, 31, 69\}$
 $U = \{22\}$

❖ 삽입 정렬 수행 과정

- ⑦ 7단계 : 현재 U의 첫째 원소 22를 S의 마지막 원소 69와 비교하면 $22 < 69$ 이므로 그 앞자리 원소 31과 비교. $22 < 31$ 이므로 그 앞자리 원소 30과 비교. $22 < 30$ 이므로 다시 그 앞자리 원소 16과 비교. $22 > 16$ 이므로 원소 16과 30 사이에 삽입. U가 공집합이 되었으므로 실행을 종료



$S = \{2, 8, 10, 16, 22, 30, 31, 69\}$
 $U = \{ \}$

❖ 크기가 n 인 배열 $a[]$ 의 원소를 삽입 정렬하는 알고리즘

알고리즘	삽입 정렬
<pre>insertionSort(a[], n) for(i ← 1; i < n; i ← i + 1) do { temp ← a[i]; j ← i; // 삽입 거리를 만들기 위해 이동할 원소 표시 if(a[j - 1] > temp) then move ← true else move ← false; while (move and j > 0) do { // 삽입 자리를 만들기 위해 a[j] ← a[j - 1]; // 삽입 자리 이후의 원소를 뒤로 이동 j ← j - 1; } a[j] ← temp // 삽입 자리에 원소 저장 } end insertionSort()</pre>	

❖ 메모리 사용공간

- n 개의 원소에 대하여 n 개의 메모리 사용

❖ 연산 시간

- 최선의 경우 : 원소들이 이미 정렬 되어있어서 비교횟수가 최소인 경우
 - 이미 정렬 되어있는 경우에는 바로 앞자리 원소와 한번만 비교
 - 전체 비교횟수 = $n-1$
 - 시간 복잡도 : $O(n)$
- 최악의 경우 : 모든 원소가 역순으로 되어있어서 비교횟수가 최대인 경우
 - 전체 비교횟수 = $1+2+3+\dots+(n-1) = n(n-1)/2$
 - 시간 복잡도 : $O(n^2)$
- 삽입 정렬의 평균 비교횟수 = $n(n-1)/4$
- 평균 시간 복잡도 : $O(n^2)$

❖ 연산 시간

- 순차 정렬된 경우 역순 정렬된 경우의 삽입 정렬 비교



- 전체 비교 횟수 : $n-1$
- 전체 자리 이동 횟수 : 0
- 시간 복잡도 : $O(n)$

(a) 순차 정렬된 경우



- 전체 비교 횟수 : $1+2+3+\dots+(n-1) = \sum_{i=1}^{n-1} i$
- 전체 자리 이동 횟수 : $1+2+3+\dots+(n-1) = \sum_{i=1}^{n-1} i$
- 실행 연산 횟수 : $\sum_{i=1}^{n-1} (\text{비교 횟수} + \text{자리 이동 횟수}) = \sum_{i=1}^{n-1} (i + i)$
- 시간 복잡도 : $O(n^2)$

(b) 역순 정렬된 경우

❖ 삽입 정렬 프로그램

```
#define _CRT_SECURE_NO_WARNINGS
#include <stdio.h>

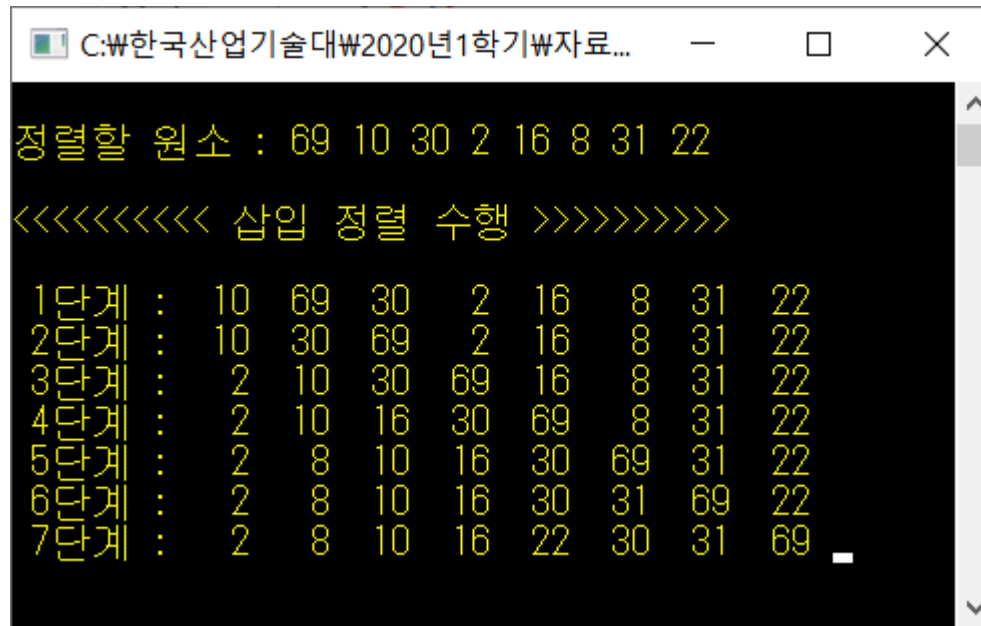
int size;
void insertionSort(int a[], int size) {
    int i, j, t, temp;
    printf("\n정렬할 원소 : ");
    for (t = 0; t < size; t++) printf("%d ", a[t]);
    printf("\n\n<<<<<<<<< 삽입 정렬 수행 >>>>>>>>>\n");
    for (i = 1; i < size; i++) {
        temp = a[i];
        j = i;
        while ((j > 0) && (a[j - 1] > temp)) {
            a[j] = a[j - 1];
            j = j - 1;
        }
        a[j] = temp;
        printf("\n %d단계 : ", i);
        for (t = 0; t < size; t++) printf("%3d ", a[t]);
    }
}
```

❖ 삽입 정렬 프로그램

```
void main() {  
    int list[8] = { 69, 10, 30, 2, 16, 8, 31, 22 };  
    size = 8;  
  
    insertionSort(list, size);  
  
    getchar();  
}
```


❖ 삽입 정렬하기 프로그램 : [교재 522p](#)

■ 실행 결과



```
C:\₩한국산업기술대₩2020년1학기₩자료...
정렬할 원소 : 69 10 30 2 16 8 31 22
<<<<<<<<< 삽입 정렬 수행 >>>>>>>>>
1단계 : 10 69 30 2 16 8 31 22
2단계 : 10 30 69 2 16 8 31 22
3단계 : 2 10 30 69 16 8 31 22
4단계 : 2 10 16 30 69 8 31 22
5단계 : 2 8 10 16 30 69 31 22
6단계 : 2 8 10 16 30 31 69 22
7단계 : 2 8 10 16 22 30 31 69
```

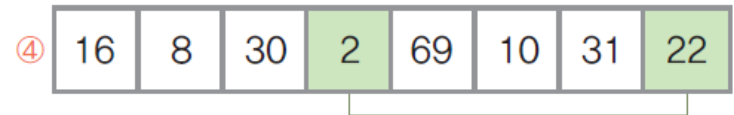
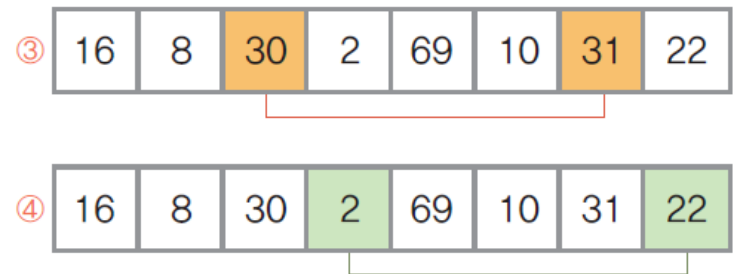
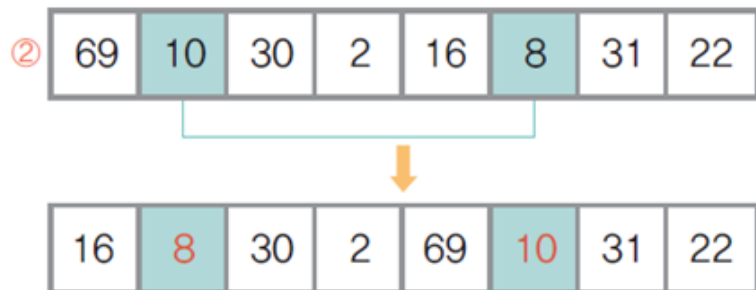
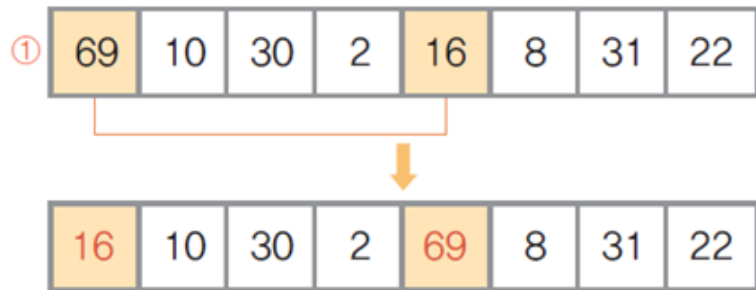
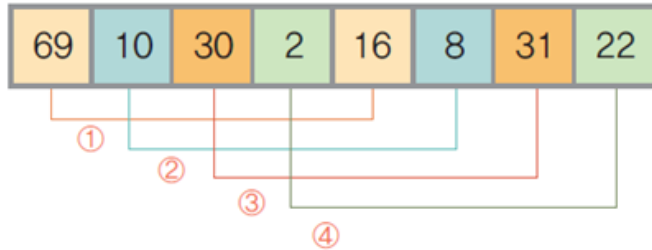
❖ 셸 정렬(shell sort)의 이해

- 일정한 간격(interval)으로 떨어져 있는 자료들끼리 부분집합을 구성하고 각 부분집합에 있는 원소들에 대해서 삽입 정렬을 수행하는 작업을 반복하면서 전체 원소들을 정렬하는 방법
 - 전체 원소에 대해서 삽입 정렬을 수행하는 것보다 부분집합으로 나누어 정렬하게 되면 비교연산과 교환연산 감소
- 셸 정렬의 부분집합
 - 부분집합의 기준이 되는 간격을 매개변수 h 에 저장
 - 한 단계가 수행될 때마다 h 의 값을 감소시키고 셸 정렬을 순환 호출
→ h 가 1이 될 때까지 반복
- 셸 정렬의 성능은 매개변수 h 의 값에 따라 달라짐
 - 정렬할 자료의 특성에 따라 매개변수 생성 함수를 사용
 - 일반적으로 사용하는 h 의 값은 원소 개수의 $1/2$ 을 사용하고 한 단계 수행될 때마다 h 의 값을 반으로 감소시키면서 반복 수행

❖ 셸 정렬(shell sort) 과정

- 정렬되지 않은 {69, 10, 30, 2, 16, 8, 31, 22}의 자료들을 셸 정렬 방법으로 정렬하는 과정
- 1) 원소 개수가 여덟 개이므로 매개변수 h 는 4에서 시작한다. $h=4$ 이므로 간격이 4만큼 떨어져 있는 원소들을 같은 부분집합으로 만들면 네 개의 부분 집합이 만들어짐
 - ① 첫 번째 부분집합 {69, 16}에 대해서 삽입 정렬을 수행하여 정렬
 - ② 두 번째 부분집합 {10, 8}에 대해서 삽입 정렬을 수행
 - ③ 세 번째 부분집합 {30, 31}에 대해서 삽입 정렬을 수행.
 $30 < 31$ 이므로 자리 이동은 이루어지지 않음.
 - ④ 네 번째 부분집합 {2, 22}에 대해서 삽입 정렬을 수행. $2 < 22$ 이므로 자리 이동은 이루어지지 않음

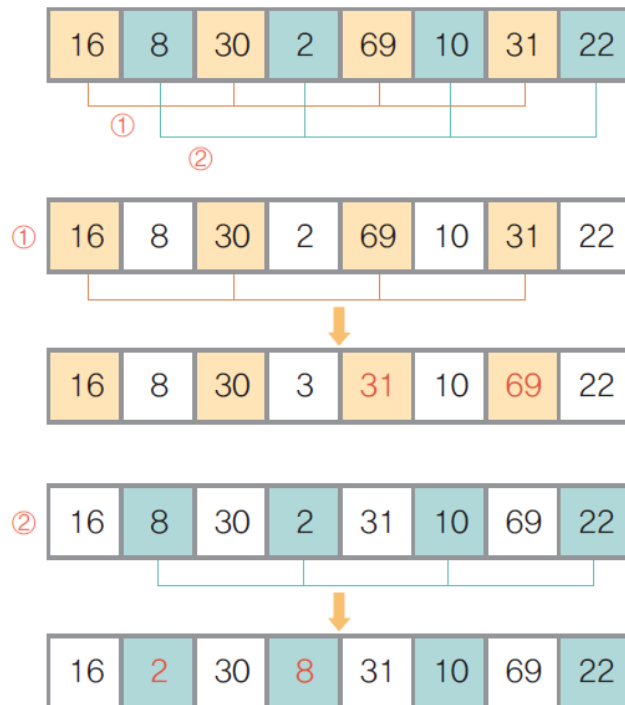
❖ 셀 정렬(shell sort) 과정



❖ 셀 정렬(shell sort) 과정

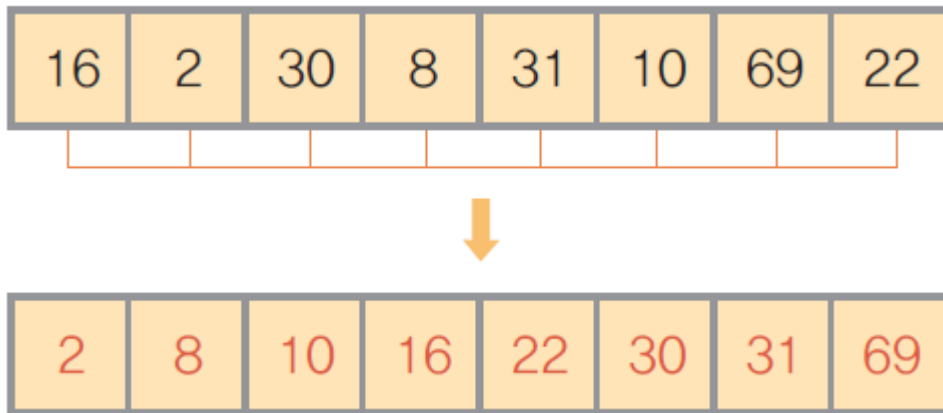
2) 이제 h 를 2로 변경하고 다시 셀 정렬을 시작. $h=2$ 이므로 간격이 2만큼 떨어진 원소들을 같은 부분집합으로 만들면 두 개의 부분집합이 만들어짐

- ① 첫 번째 부분집합 {16, 30, 69, 31}에 대해 삽입 정렬을 수행하여 정렬
- ② 두 번째 부분집합 {8, 2, 10, 22}에 대해 삽입 정렬을 수행하여 정렬



❖ 셀 정렬(shell sort) 과정

- 3) 이제 h 를 1로 변경하고 다시 셀 정렬을 시작. $h=1$ 이므로 간격이 1만큼 떨어져 있는 원소들을 같은 부분집합으로 만들면 한 개의 부분집합이 만들어짐. 즉 전체 원소에 대해서 삽입 정렬을 수행



❖ 셸 정렬 알고리즘

알고리즘	셸 정렬
<pre>shellSort(a[], n) interval ← n; while (interval ≥ 1) do { interval ← interval/2; for (i ← 0; i < interval; i ← i+1) do { intervalSort(a[], i, n, interval); } } end shellSort()</pre>	

❖ 셀 정렬 알고리즘

알고리즘

셀 부분집합에서의 삽입정렬

```
intervalSort(a[ ], begin, end, interval)
  for (i ← begin+interval; i ≤ end; i ← i+interval) do {
    item ← a[i];
    for (j ← i-interval; j ≥ begin and item < a[j]; j ← j-interval) do
      a[j+interval] ← a[j];
    a[j+interval] ← item;
  }
end intervalSort( )
```


❖ 셀 정렬의 특징

- 메모리 사용공간
 - n 개의 원소에 대하여 n 개의 메모리와 매개변수 h 에 대한 저장 공간 사용
- 연산 시간
 - 비교횟수
 - 처음 원소의 상태에 상관없이 매개변수 h 에 의해 결정
 - 일반적인 시간 복잡도 : $O(n^{1.25})$
 - 셀 정렬은 삽입 정렬의 시간 복잡도 $O(n^2)$ 보다 개선된 정렬 방법

❖ 셀 정렬하기 프로그램

```
#include <stdio.h>

typedef int element;
int size;

// 부분집합에 대해 셀 정렬을 수행하는 연산
void intervalSort(element a[], int begin, int end, int interval) {
    int i, j;
    element item;
    for (i = begin + interval; i <= end; i = i + interval) {
        item = a[i];
        for (j = i - interval; j >= begin && item < a[j]; j = j - interval)
            a[j + interval] = a[j];
        a[j + interval] = item;
    }
}
```

❖ 셀 정렬하기 프로그램

```
void shellSort(int a[], int size)
{
    int i, t, interval;
    printf("\n정렬할 원소 : ");
    for (t = 0; t < size; t++) printf("%d ", a[t]);
    printf("\n\n<<<<<<<<< 셀 정렬 수행 >>>>>>>>>\n");
    interval = size / 2;
    while (interval >= 1) {
        for (i = 0; i < interval; i++) intervalSort(a, i, size - 1, interval);
        printf("\n interval=%d >> ", interval);
        for (t = 0; t < size; t++) printf("%d ", a[t]);
        printf("\n");
        interval = interval / 2;
    }
}
```

❖ 셀 정렬하기 프로그램

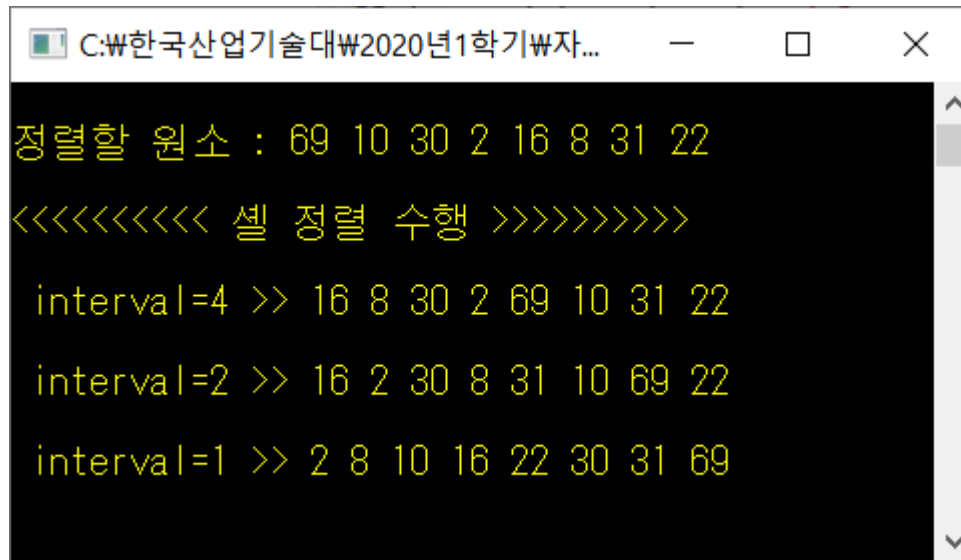
```
void main()
{
    int list[8] = { 69, 10, 30, 2, 16, 8, 31, 22 };
    size = 8;

    shellSort(list, size);

    getchar();
}
```

❖ 셀 정렬하기 프로그램 : 교재 525p

■ 실행 결과



```
C:\₩한국산업기술대₩2020년1학기₩자...
정렬할 원소 : 69 10 30 2 16 8 31 22
<<<<<<<<< 셀 정렬 수행 >>>>>>>>>
interval=4 >> 16 8 30 2 69 10 31 22
interval=2 >> 16 2 30 8 31 10 69 22
interval=1 >> 2 8 10 16 22 30 31 69
```

❖ 퀵 정렬(quick sort)

- 정렬할 전체 원소에 대해서 정렬을 수행하지 않고, 기준 값을 중심으로 왼쪽 부분 집합과 오른쪽 부분 집합으로 분할하여 정렬하는 방법
 - 왼쪽 부분 집합에는 기준 값보다 작은 원소들을 이동시키고, 오른쪽 부분 집합에는 기준 값보다 큰 원소들을 이동시킴
 - 기준 값 : 피벗(pivot)
 - 일반적으로 전체 원소 중에서 가운데에 위치한 원소를 선택
- 퀵 정렬은 다음의 두 가지 기본 작업을 반복 수행하여 완성
 - 분할(divide)
 - 정렬할 자료들을 기준 값을 중심으로 두 개로 나눠 부분집합을 만듦
 - 정복(conquer)
 - 부분집합 안에서 기준 값의 정렬 위치를 정함

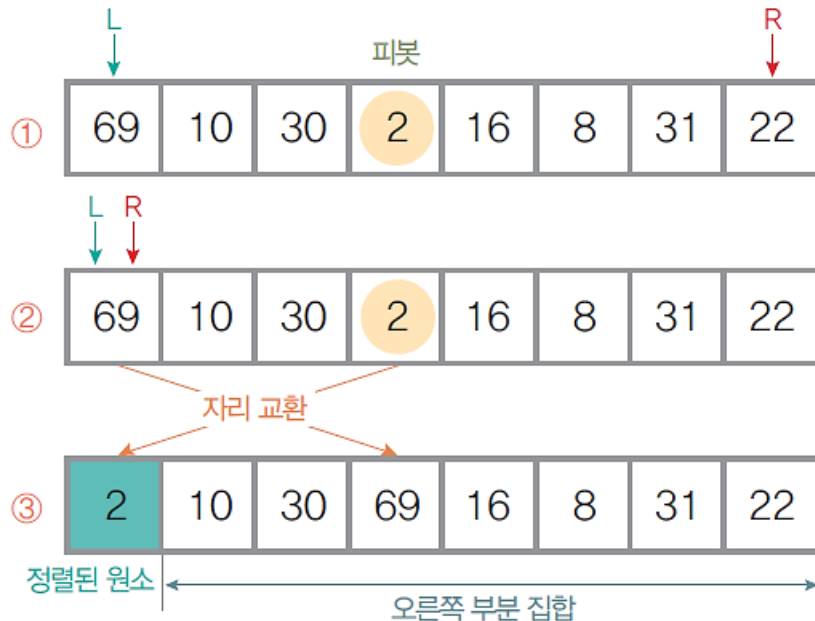
❖ 퀵 정렬(quick sort)

■ 퀵 정렬 동작 규칙

- ① 왼쪽 끝에서 오른쪽으로 움직이면서 크기를 비교하여 피벗 보다 크거나 같은 원소를 찾아 L로 표시한다. 단 L은 R과 만나면 더 이상 오른쪽으로 이동하지 못하고 멈춘다.
- ② 오른쪽 끝에서 왼쪽으로 움직이면서 피벗 보다 작은 원소를 찾아 R로 표시한다. 단, R은 L과 만나면 더 이상 왼쪽으로 이동하지 못하고 멈춘다.
- ③ -**㉠** ①과 ②에서 찾은 L 원소와 R 원소가 있는 경우, 서로 교환하고 L과 R의 현재 위치에서 ①과 ② 작업을 다시 수행한다.
- ③ -**㉢** ①~②를 수행하면서 L과 R이 같은 원소에서 만나 멈춘 경우, 피벗과 R의 원소를 서로 교환한다. 교환된 자리를 피벗 위치로 정하고 현재 단계의 피벗 정렬을 끝낸다.
- ④ 피벗의 확정된 위치를 기준으로 만들어진 새로운 왼쪽 부분집합과 오른쪽 부분집합에 대해서 ①~③의 퀵 정렬을 순환적으로 반복 수행하는데, 모든 부분집합의 크기가 1 이하가 되면 전체 퀵 정렬을 종료한다.

❖ 퀵 정렬(quick sort) 수행과정

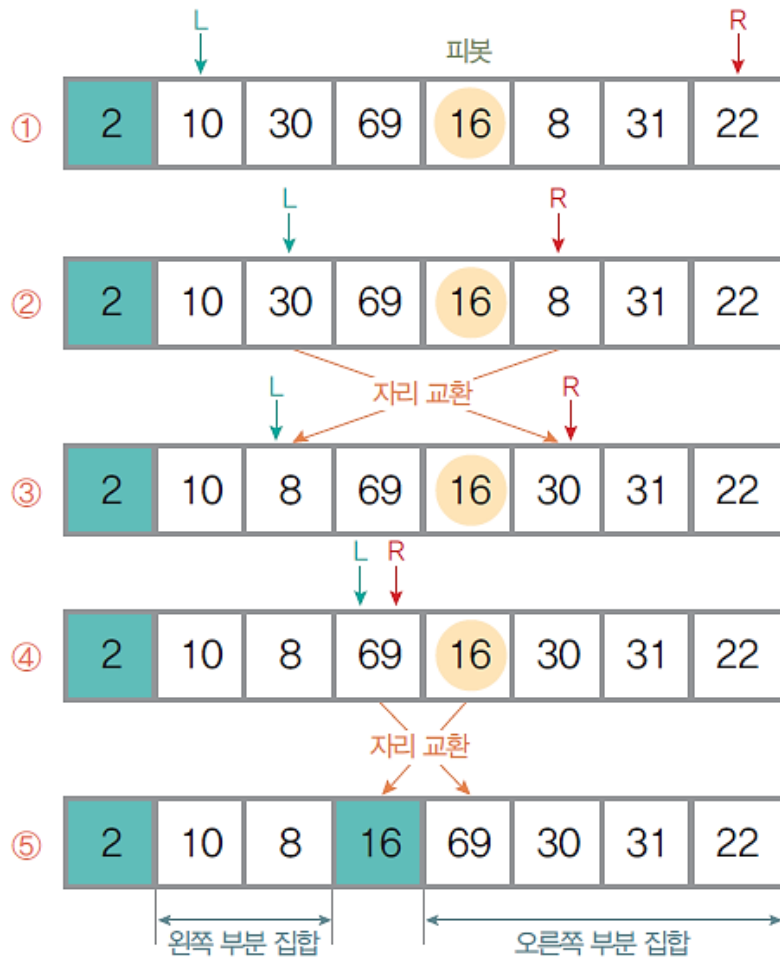
- 정렬하지 않은 {69, 10, 30, 2, 16, 8, 31, 22} 자료를 퀵 정렬 방법으로 정렬하는 과정 (가운데 원소를 피봇으로 정함)



1단계

- 원소 2를 피봇으로 선택하고 퀵 정렬을 시작한다.
- L은 왼쪽 끝에서 오른쪽으로 움직이면서 피봇보다 크거나 같은 원소를 찾고, R은 오른쪽 끝에서 왼쪽으로 움직이면서 피봇보다 작은 원소를 찾는다. L은 원소 69를 찾았지만, R은 피봇보다 작은 원소를 찾지 못한 상태로 원소 69에서 L과 만나게 된다.
- L과 R이 만났으므로 원소 69를 피봇과 자리를 교환하고 피봇 원소 2의 위치를 확정한다.

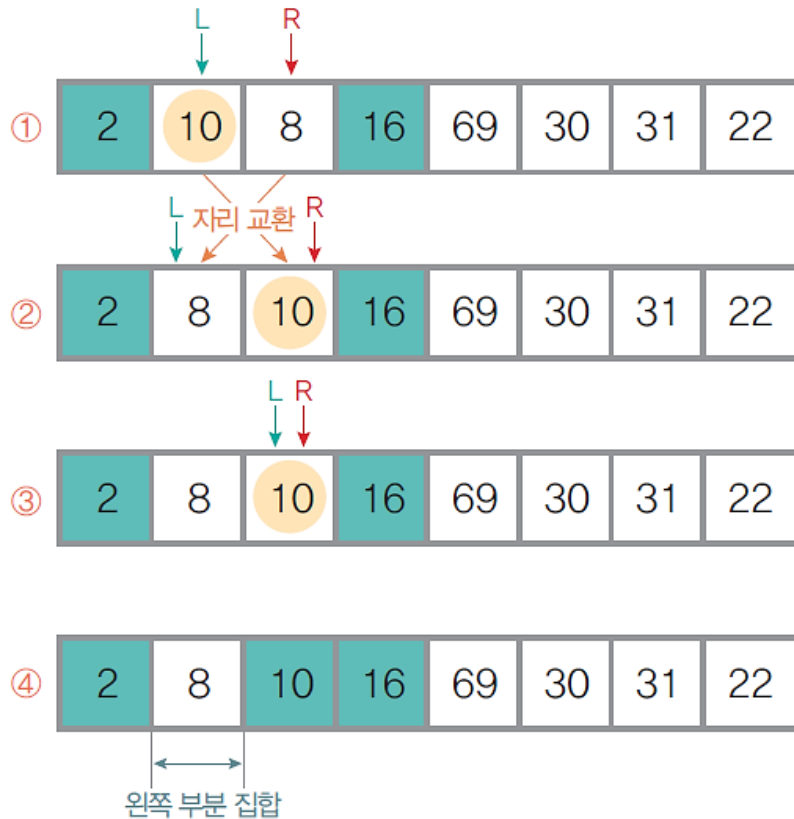
❖ 퀵 정렬(quick sort) 수행과정



2단계 : 피봇 2의 왼쪽 부분집합은 공집합이므로 퀵 정렬을 수행하지 않고, 오른쪽 부분집합에 대해서 퀵 정렬을 수행.

- ① 오른쪽 부분집합의 원소가 7개이므로 가운데 있는 원소 16을 피봇으로 선택
- ② L은 오른쪽으로 움직이면서 피봇보다 크거나 같은 원소인 30을 찾고, R은 왼쪽으로 움직이면서 피봇보다 작은 원소인 8을 찾는다.
- ③ L이 찾은 30과 R이 찾은 8을 서로 자리를 교환한 후 현재 위치에서 L은 다시 오른쪽으로 움직이면서 피봇보다 크거나 같은 원소 69를 찾고 R은 피봇보다 작은 원소를 찾는다.
- ④ R이 원소 69에서 L과 만나 더 이상 진행할 수 없는 상태가 되었으므로,
- ⑤ 원소 69를 피봇과 교환하고 피봇 원소 16의 위치를 확정한다.

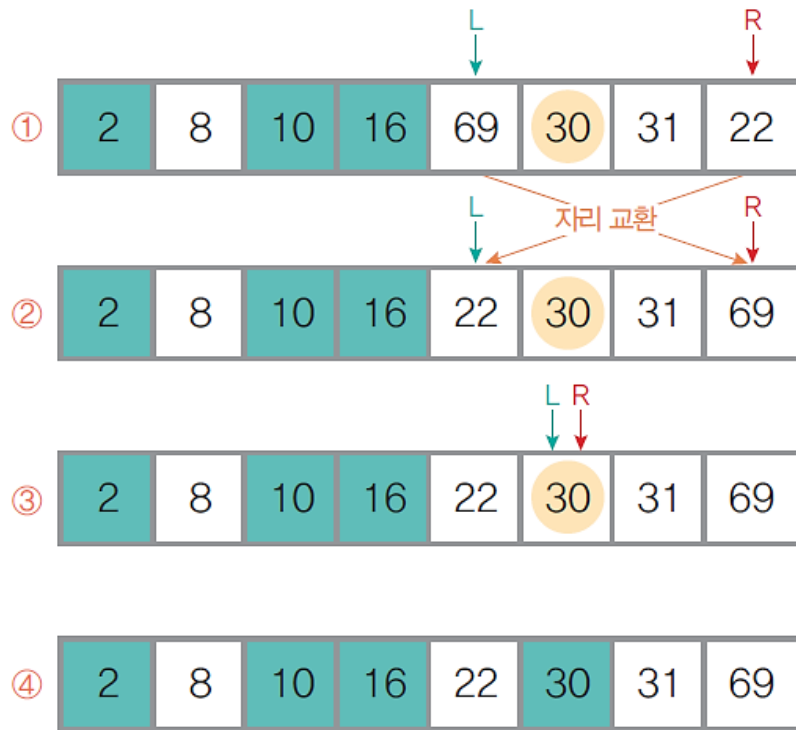
❖ 퀵 정렬(quick sort) 수행과정



3단계 : 위치가 확정된 피벗 16을 기준으로 새로 생긴 왼쪽 부분집합에서 퀵 정렬을 수행

- ① 원소 10을 피벗으로 선택하고 L은 오른쪽으로 움직이면서 피벗보다 크거나 같은 원소를, R은 왼쪽으로 움직이면서 피벗보다 작은 원소를 찾음
- ② L이 찾은 10과 R이 찾은 8을 서로 교환
- ③ 현재 위치에서 L은 다시 오른쪽으로 움직이다가 원소 10에서 R과 만나서 멈추게 됨. 더 이상 진행할 수 없는 상태가 되었으므로,
- ④ R의 원소와 피벗을 교환하고 피벗 원소 10의 위치를 확정한다(이 경우에는 R과 피벗 위치가 같았으므로, 자리를 교환하기 전과 후의 상태가 같음)

❖ 퀵 정렬(quick sort) 수행과정



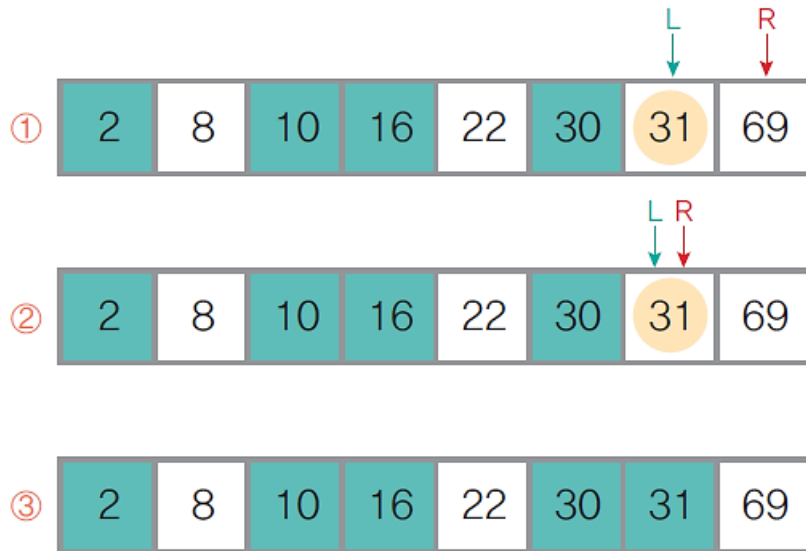
4단계 : 피벗 10의 왼쪽 부분집합은 원소가 한 개이므로 퀵 정렬을 수행하지 않고, 오른쪽 부분집합은 공집합이므로 역시 퀵 정렬을 수행하지 않음
이제 [2]에서 피벗이었던 원소 16의 오른쪽 부분집합에 대해서 퀵 정렬을 수행

- ① 오른쪽 부분집합의 원소가 네 개이므로 가운데 있는 원소 30을 피벗으로 선택. L은 오른쪽으로 이동하면서 피벗보다 크거나 같은 원소를 찾고, R은 왼쪽으로 움직이면서 피벗보다 작은 원소를 찾음
- ② L이 찾은 69와 R이 찾은 22를 서로 교환. 현재 위치에서 다시 L은 피벗보다 크거나 같은 원소를 찾아 오른쪽으로 움직이고 R은 피벗보다 작은 원소를 찾아 왼쪽으로 움직이다가
- ③ 원소 30에서 L과 R이 만나 멈춤. 더 이상 진행할 수 없으므로
- ④ R의 원소와 피벗을 교환하고 피벗 원소 30의 위치가 확정 (이 경우에 R과 피벗 위치가 같았으므로, 자리를 교환하기 전과 후의 상태가 같음)

❖ 퀵 정렬(quick sort) 수행과정

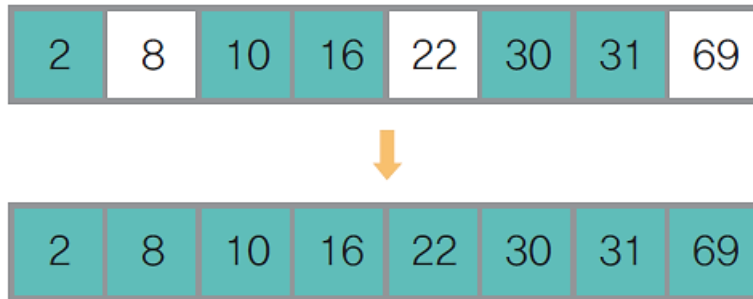
5단계 : 피벗 30의 왼쪽 부분집합의 원소가 한 개이므로 퀵 정렬을 수행하지 않고, 오른쪽 부분집합에 대해서 퀵 정렬을 수행

- ① 이때 피벗은 31을 선택하고 L은 오른쪽으로 움직이면서 피벗보다 크거나 같은 원소를 찾고 R은 왼쪽으로 움직이면서 피벗보다 작은 원소를 찾음
- ② L과 R이 원소 31에서 만나 더 이상 진행하지 못하는 상태가 되어
- ③ 원소 31을 피벗과 교환하여 위치를 확정(이 경우에는 R과 피벗 위치가 같았으므로, 자리를 교환하기 전과 후의 상태가 같음).



❖ 퀵 정렬(quick sort) 수행과정

6단계 : 피벗 31의 오른쪽 부분집합의 원소가 한 개이므로 퀵 정렬을 수행하지 않는다. 모든 부분 집합의 크기가 1 이하이므로 전체 퀵 정렬을 종료



❖ 퀵 정렬(quick sort) 알고리즘

- 크기가 n 인 배열 $a[]$ 의 퀵 정렬하는 알고리즘

알고리즘	퀵 정렬
<pre>quickSort(a[], begin, end) if (m < n) then { p ← partition(a, begin, end); quickSort(a[], begin, p-1); quickSort(a[], p+1, end); } end quickSort()</pre>	

❖ 퀵 정렬(quick sort) 알고리즘

- 퀵 정렬은 배열 $a[]$ 를 두 개로 분할하는 `partition( )` 연산이 필요

알고리즘	파티션 분할
<pre>partiton(a[], begin, end) pivot ← (begin + end)/2; L ← begin; R ← end; while(L<R) do { while(a[L]<a[pivot] and L<R) do L++; while(a[R]≥a[pivot] and L<R) do R--; if(L<R) then { // L의 원소와 R의 원소 교환 temp ← a[L]; a[L] ← a[R]; a[R] ← temp; } } temp ← a[pivot]; // R의 원소와 피벗 원소 교환 a[pivot] ← a[R]; a[R] ← temp; return L; end partition()</pre>	

❖ 메모리 사용공간

- n 개의 원소에 대하여 n 개의 메모리 사용

❖ 연산 시간

- 최선의 경우
 - 피봇에 의해서 원소들이 왼쪽 부분 집합과 오른쪽 부분 집합으로 정확히 $n/2$ 개씩 2등분이 되는 경우가 반복되어 수행 단계 수가 최소가 되는 경우
- 최악의 경우
 - 피봇에 의해 원소들을 분할하였을 때 한개와 $n-1$ 개로 한쪽으로 치우쳐 분할되는 경우가 반복되어 수행 단계 수가 최대가 되는 경우
- 평균 시간 복잡도 : $O(n \log_2 n)$
 - 같은 시간 복잡도를 가지는 다른 정렬 방법에 비해서 자리 교환 횟수를 줄임으로써 더 빨리 실행되어 실행 시간 성능이 좋은 정렬 방법임.

❖ 퀵 정렬하기 프로그램

```
#define _CRT_SECURE_NO_WARNING
#include "stdio.h"
typedef int element;
int size, i = 0;

// 주어진 부분집합 안에서 피벗의 위치를確定하여 분할 위치를 정하는 연산
int partition(element a[], int begin, int end)
{
    int pivot, L, R, t;
    element temp;
    L = begin;
    R = end;
    pivot = (int)((begin + end) / 2.0); // 중간에 위치한 원소를 피벗 원소로 선택
```

❖ 퀵 정렬하기 프로그램

```
printf("Wn [%d단계 : pivot=%d ] Wn", ++i, a[pivot]);  
while (L<R)  
{  
    while ((a[L]<a[pivot]) && (L<R))  
        L++;  
    while ((a[R] >= a[pivot]) && (L<R))  
        R--;  
    if (L<R)                // L과 R 원소의 자리 교환  
    {  
        temp = a[L];  
        a[L] = a[R];  
        a[R] = temp;  
  
        if (L == pivot)      // L이 피벗인 경우,  
            pivot = R;      // 변경된 피벗의 위치(R)를 저장!  
    }                        // if(L<R)  
}                            // while(L<R)
```

❖ 퀵 정렬하기 프로그램

```
// (L=R)이 된 경우,  
// 더 이상 진행할 수 없으므로 R 원소와 피벗 원소의 자리를 교환하여 마무리  
temp = a[pivot];  
a[pivot] = a[R];  
a[R] = temp;  
for (t = 0; t < size; t++)  
    printf(" %d", a[t]);    // 현재의 정렬 상태 출력  
printf("\n");  
return R;                  // 정렬되어 확정된 피벗의 위치 반환  
}
```

❖ 퀵 정렬하기 프로그램

```
void quickSort(element a[], int begin, int end)
{
    int p;
    if (begin < end)
    {
        // 피봇의 위치에 의해 분할 위치 결정
        p = partition(a, begin, end);
        // 피봇의 왼쪽 부분집합에 대해 퀵 정렬을 재귀호출
        quickSort(a, begin, p - 1);
        // 피봇의 오른쪽 부분집합에 대해 퀵 정렬을 재귀호출
        quickSort(a, p + 1, end);
    }
}
```

❖ 퀵 정렬하기 프로그램

```
void main()
{
    element list[8] = { 69, 10, 30, 2, 16, 8, 31, 22 };
    size = 8;          // list 배열의 원소 개수
    printf("\n [ 초기 상태 ] \n");
    for (int i = 0; i <= size - 1; i++)
        printf(" %d", list[i]);
    printf("\n");

    quickSort(list, 0, size - 1);

    getchar();
}
```

❖ 퀵 정렬하기 프로그램 : 교재 514p

■ 실행 결과

```
[ 초기 상태 ]
69 10 30 2 16 8 31 22

[1단계 : pivot=2 ]
2 10 30 69 16 8 31 22

[2단계 : pivot=16 ]
2 10 8 16 69 30 31 22

[3단계 : pivot=10 ]
2 8 10 16 69 30 31 22

[4단계 : pivot=30 ]
2 8 10 16 22 30 31 69

[5단계 : pivot=31 ]
2 8 10 16 22 30 31 69
```

❖ 힉프 정렬(heap sort)의 이해

- 힉프 자료구조를 이용한 정렬 방법
- 힉프에서는 항상 가장 큰 원소가 루트 노드가 되고 삭제 연산을 수행하면 항상 루트 노드의 원소를 삭제하여 반환
 - 최대 힉프에 대해서 원소의 개수만큼 삭제 연산을 수행하여 내림차순으로 정렬 수행
 - 최소 힉프에 대해서 원소의 개수만큼 삭제 연산을 수행하여 오름차순으로 정렬 수행

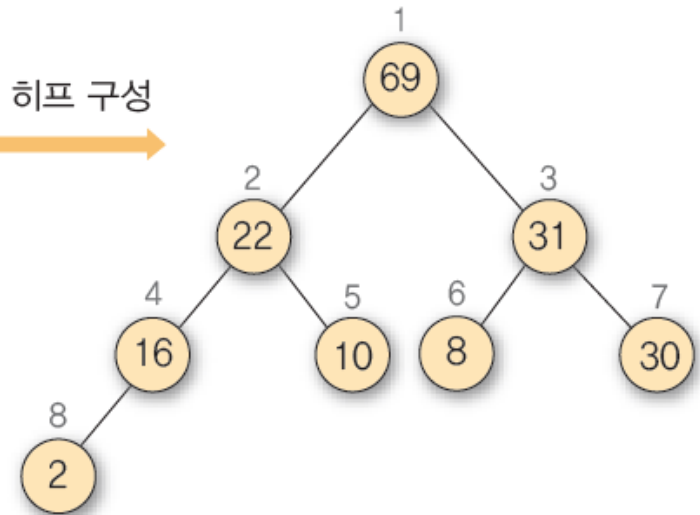
❖ 정렬 과정

- 정렬되지 않은 {69, 10, 30, 2, 16, 8, 31, 22}의 자료들을 히프 정렬 방법으로 정렬하는 과정

(1) 초기 상태 : 정렬할 원소에 대해 7장에서 설명한 삽입 연산을 이용해 최대 히프를 구성

[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]	[8]
...	69	10	30	2	16	8	31	22

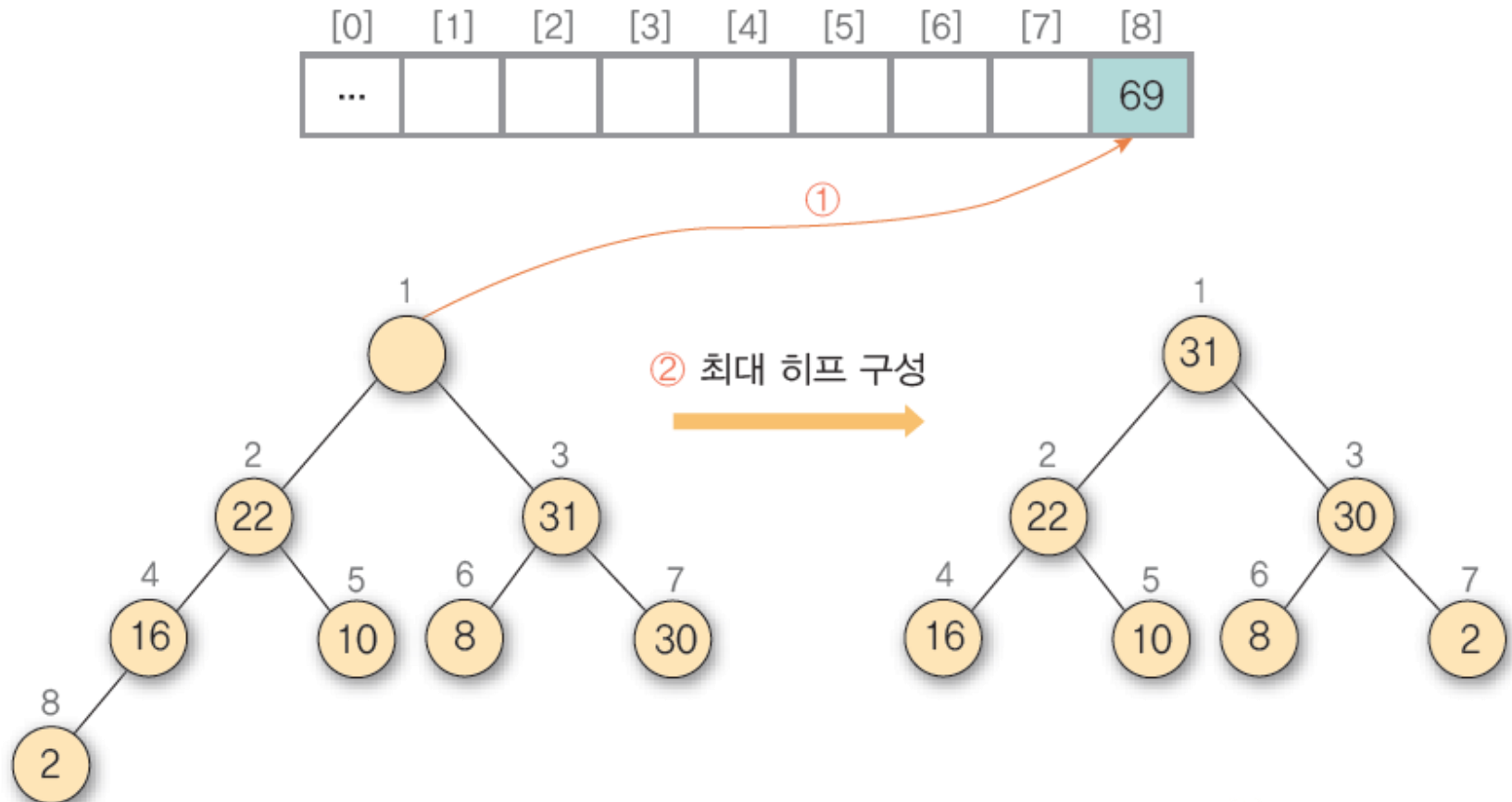
최대 히프 구성



```
C:\한국산업기술대\2020년1학기\자...
정렬할 원소 : 69 10 30 2 16 8 31 22
<<<<<<<<< 히프 정렬 수행 >>>>>>>>>
Heap   : 69 22 31 16 10 8 30 2 _
```

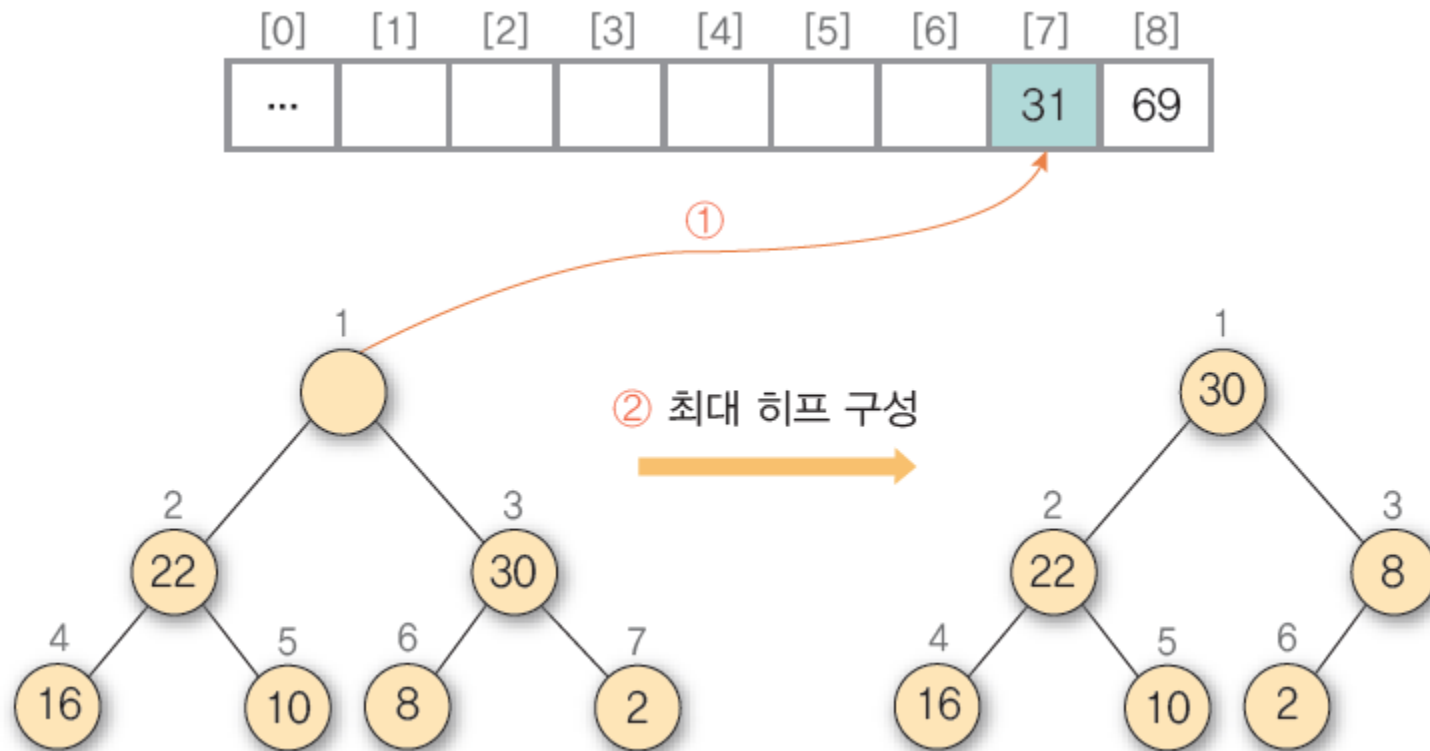

❖ 정렬 과정

(2) 초기 상태 : ① 힉프에 삭제 연산을 수행하여 루트 노드의 원소 69를 구한 후 배열의 마지막 자리에 저장 ② 나머지 힉프를 최대 힉프로 재구성. 원소를 삭제하 힉프를 재구성



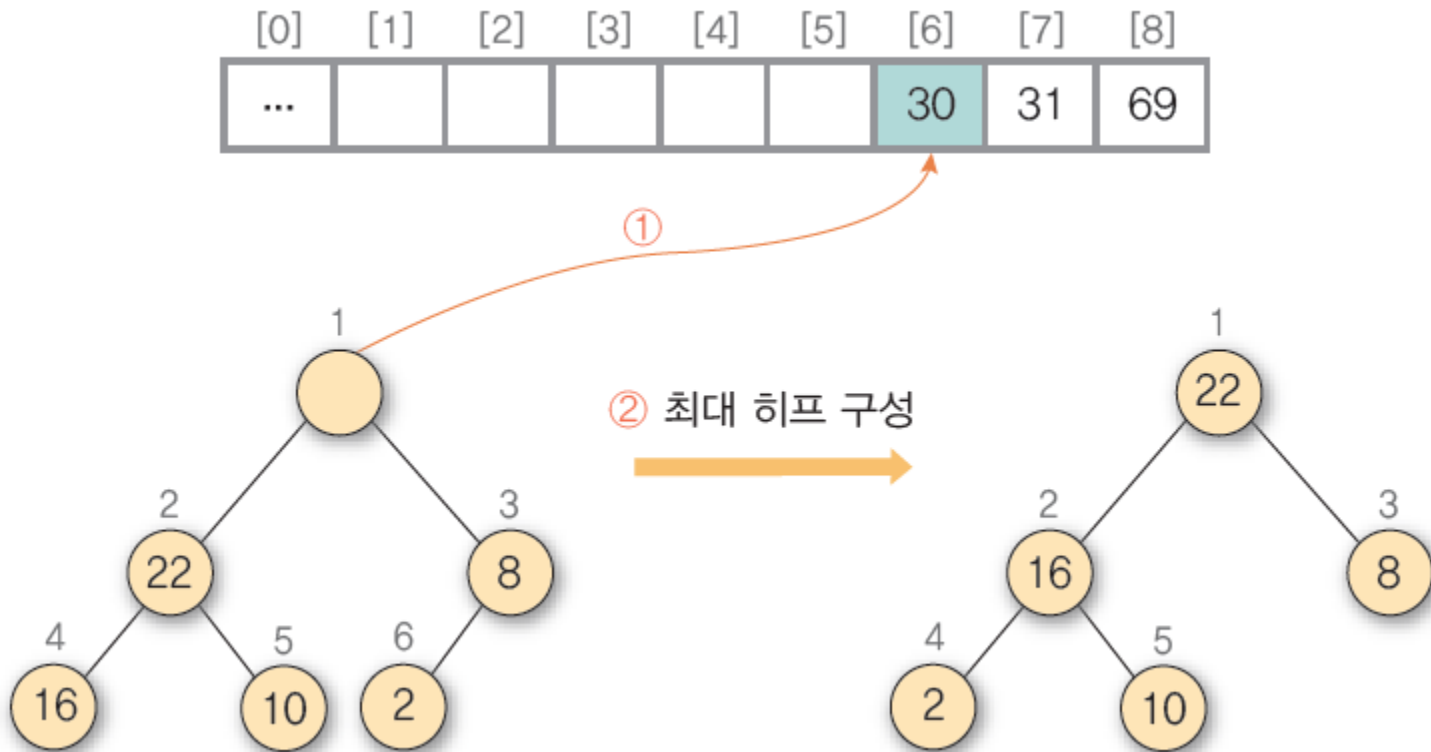
❖ 정렬 과정

(3) ① 히프에 삭제 연산을 수행하여 루트 노드의 원소 31을 구한 후 배열의 비어 있는 마지막 자리에 저장 ② 나머지 히프를 최대 히프로 재구성



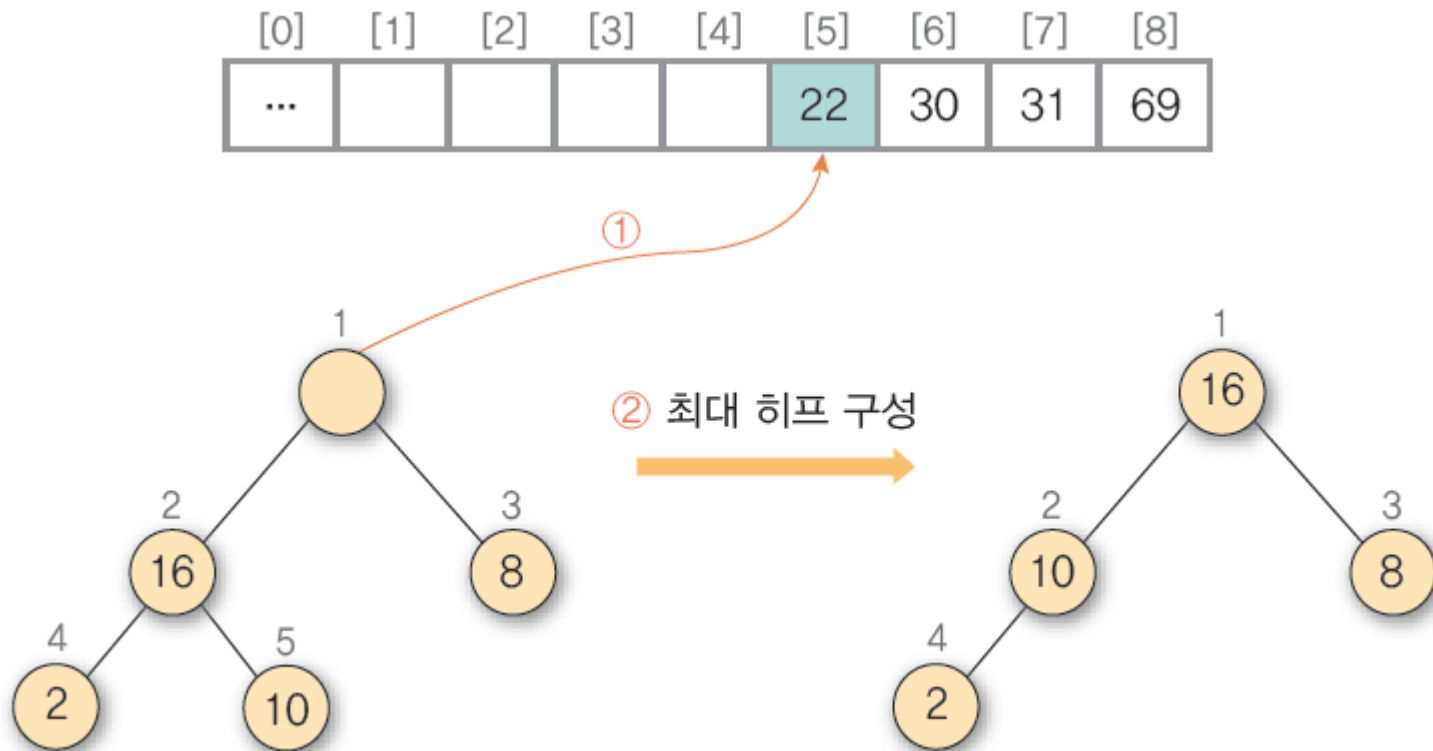
❖ 정렬 과정

(4) ① 히프에 삭제 연산을 수행하여 루트 노드의 원소 30을 구한 후 배열의 비어 있는 마지막 자리에 저장 ② 나머지 히프를 최대 히프로 재구성



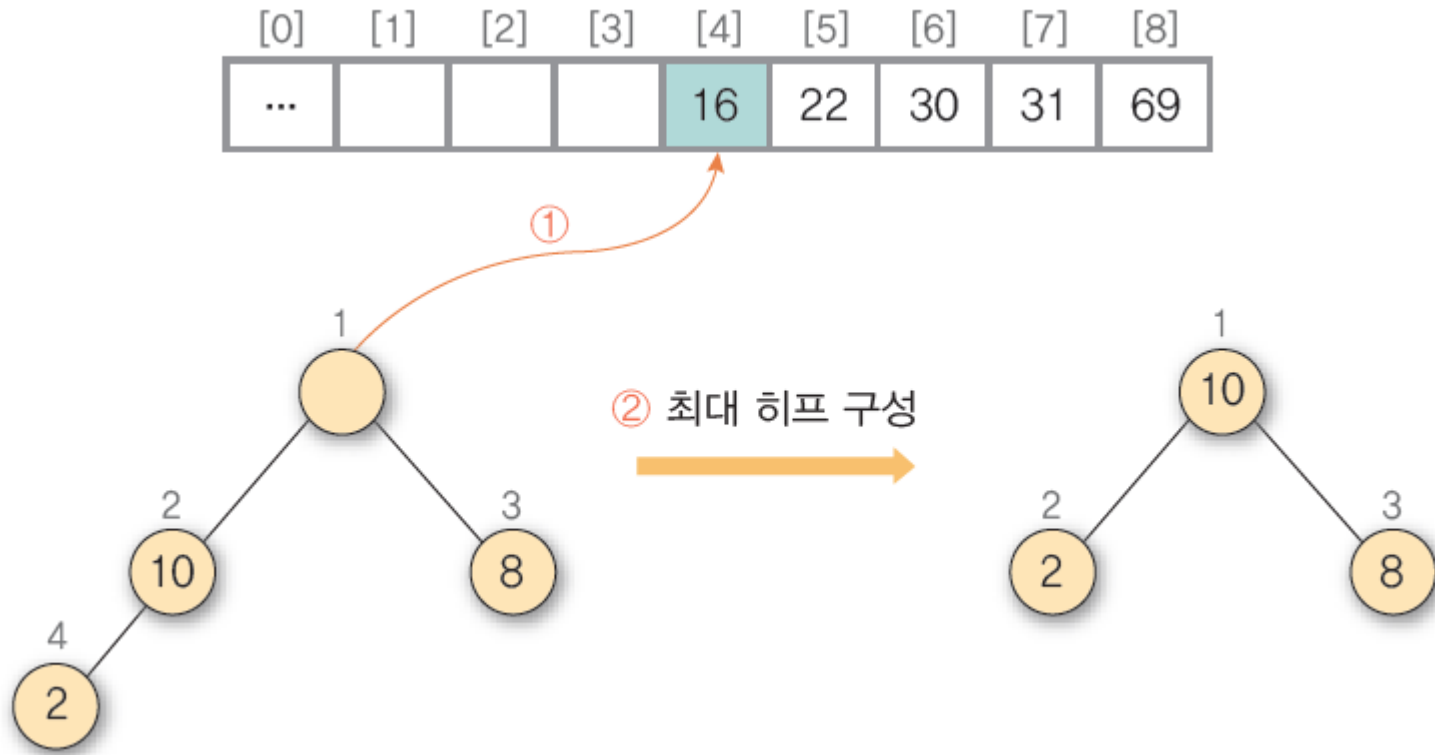
❖ 정렬 과정

(5) ① 히프에 삭제 연산을 수행하여 루트 노드의 원소 22를 구한 후 배열의 비어 있는 마지막 자리에 저장 ② 나머지 히프를 최대 히프로 재구성



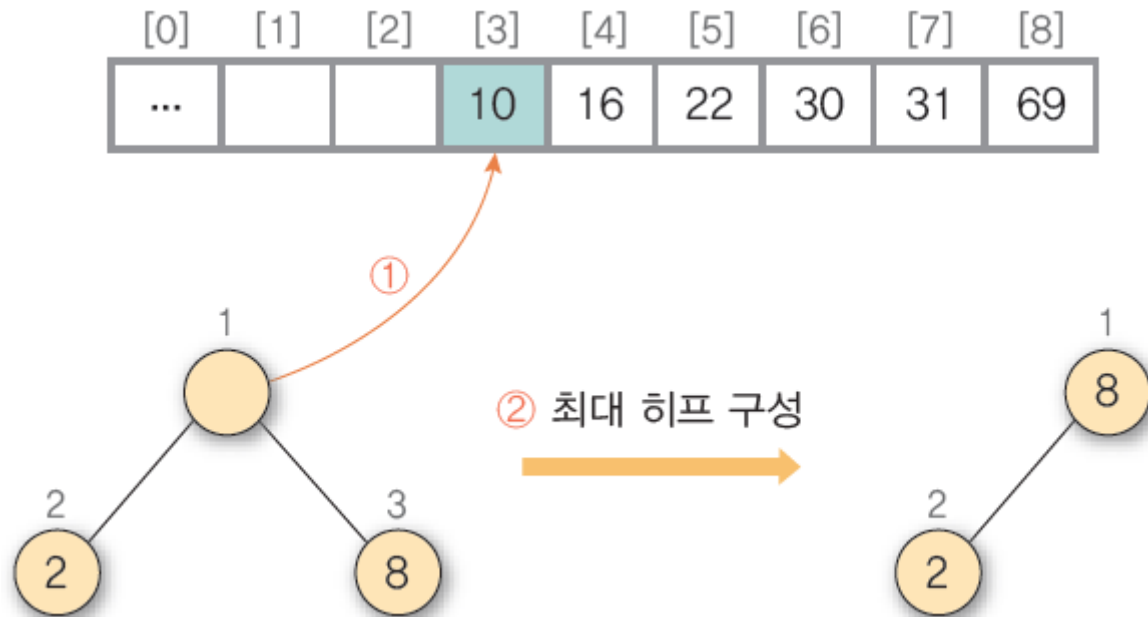
❖ 정렬 과정

(6) ① 힙에 삭제 연산을 수행하여 루트 노드의 원소 16을 구한 후 배열의 비어 있는 마지막 자리에 저장 ② 나머지 힙을 최대 힙으로 재구성



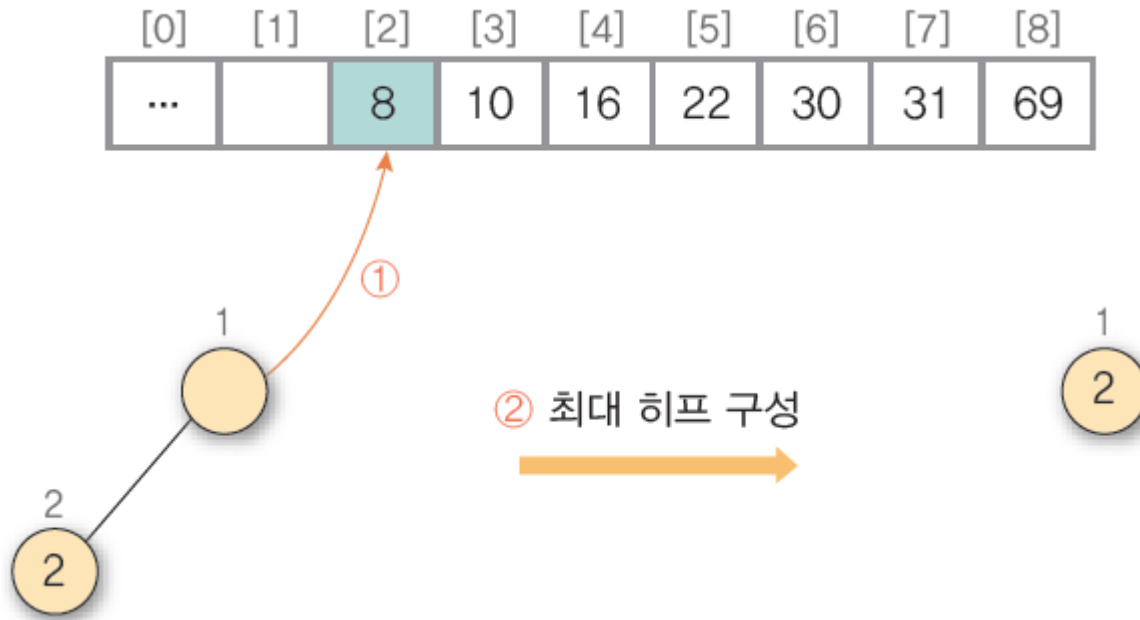
❖ 정렬 과정

(7) ① 힉프에 삭제 연산을 수행하여 루트 노드의 원소 10을 구한 후 배열의 비어 있는 마지막 자리에 저장 ② 나머지 힉프를 최대 힉프로 재구성



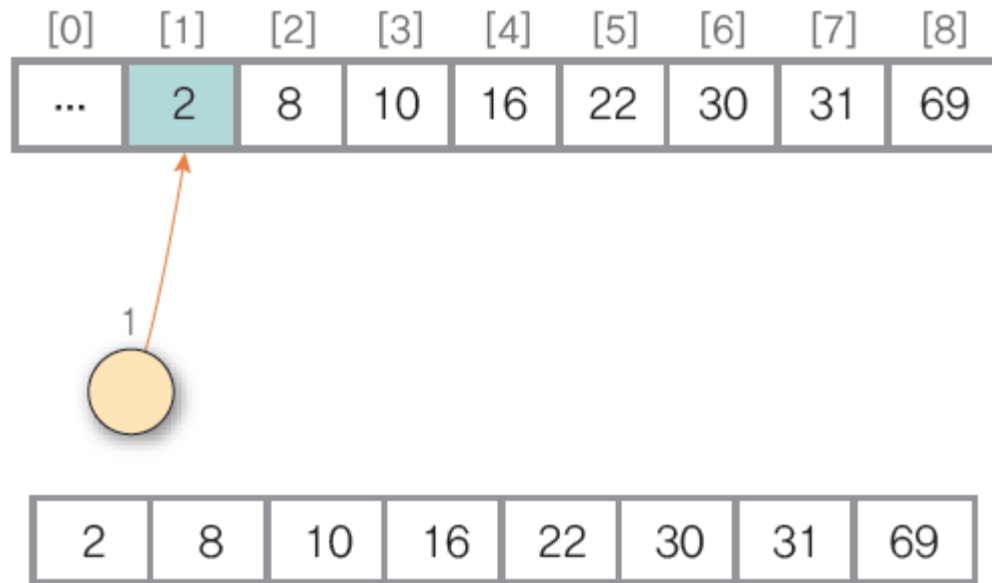
❖ 정렬 과정

(8) ① 힉프에 삭제 연산을 수행하여 루트 노드의 원소 8을 구한 후 배열의 비어 있는 마지막 자리에 저장 ② 나머지 힉프를 최대 힉프로 재구성



❖ 정렬 과정

(9)히프에 삭제 연산을 수행하여 루트 노드의 원소 2를 구한 후 배열의 비어 있는 마지막 자리에 저장. 나머지 히프를 최대 히프로 재구성해야 하는데 공백 히프가 되었으므로 히프 정렬을 종료



❖ 히프 정렬 알고리즘

알고리즘	히프 정렬
<pre>heapSort(a[]) n ← a.length - 1; for (i ← n/2; i ≥ 1; i ← i-1) do // 배열 a[]를 히프로 변환 makeHeap(a, i, n) for (i ← n-1 ; i ≥ 1; i ← i-1) do { temp ← a[i+1]; // 히프의 루트 노드 원소를 a[1] ← a[i+1]; // 배열의 비어 있는 a[i+1] ← temp // 마지막 자리에 저장 amkeHeap(a, 1, i); } end heapSort()</pre>	

❖ 배열을 히프 구조로 재구성하는 makeHeap() 연산에 대한 알고리즘

알고리즘	히프 재구성
<pre>makeHeap(a[], h, m) for (j ← 2*h; j ≤ m; j ← 2*j) do { if (j < m) then if(a[j] < a[j+1] then j ← j+1; if(a[h] ≥ a[j] then exit; else a[j/2] ← a[j]; } a[j/2] ← a[h]; end heapSort()</pre>	

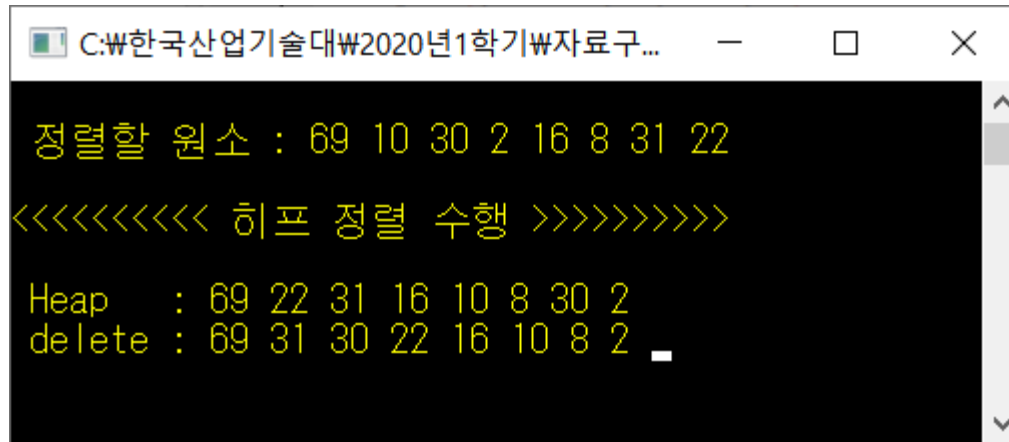
❖ 메모리 사용공간

- 원소 n 개에 대해서 n 개의 메모리 공간 사용
- 크기 n 의 힉프 저장 공간

❖ 연산 시간

- 힉프 재구성 연산 시간
 - n 개의 노드에 대해서 완전 이진 트리는 $\log_2(n+1)$ 의 레벨을 가지므로 완전 이진 트리를 힉프로 구성하는 평균시간은 $O(\log_2 n)$
 - n 개의 노드에 대해서 n 번의 힉프 재구성 작업 수행
- 평균 시간 복잡도 : $O(n \log_2 n)$

❖ 힉프 정렬하기 프로그램 실행 결과



```
C:\₩한국산업기술대₩2020년1학기₩자료구...

정렬할 원소 : 69 10 30 2 16 8 31 22
<<<<<<<< 힉프 정렬 수행 >>>>>>>>>
Heap : 69 22 31 16 10 8 30 2
delete : 69 31 30 22 16 10 8 2
```

❖ 트리 정렬(tree sort)의 이해

- 7장의 이진 탐색 트리를 이용하여 정렬하는 방법
- 트리 정렬 수행 방법
 - 정렬할 원소들을 이진 탐색 트리로 구성
 - 이진 탐색 트리를 중위 우선 순회 함
 - 중위 순회 경로가 오름차순 정렬이 됨

❖ 트리 정렬 방법

- 정렬되지 않은 {69, 10, 30, 2, 16, 8, 31, 22}의 자료들을 트리 정렬 방법으로 정렬하는 과정

- 정렬할 원소 여덟 개를 차례대로 사입하여 이진 트리를 구성
- 이진 탐색 트리를 중위 순회 방법으로 순회하면서 원소를 저장



❖ 트리 정렬 알고리즘

- 이진 탐색 트리에서의 삽입 연산 insert()와 중위 순회 연산 사용

알고리즘	트리 정렬
<pre>treeSort(a[], n) for (i ← 0; i < n; i ← i+1) do insert(BST, a[i]); // 이진 탐색 트리의 삽입 연산 inorder(BST); // 중위 순회 연산 end treeSort()</pre>	

❖ 메모리 사용공간

- 원소 n 개에 대해서 n 개의 메모리 공간 사용
- 크기 n 의 이진 탐색 트리 저장 공간

❖ 연산 시간

- 노드 한 개에 대한 이진 탐색 트리 구성 시간 : $O(\log_2 n)$
- n 개의 노드에 대한 시간 복잡도 : $O(n \log_2 n)$

❖ 트리 정렬 예제 프로그램

```
#include <stdio.h>
#include <stdlib.h>

typedef int element;      // 이진 탐색 트리 element의 자료형을 int로 정의
typedef struct treeNode {
    element key;          // 데이터 필드
    struct treeNode* left; // 왼쪽 서브 트리 링크 필드
    struct treeNode* right; // 오른쪽 서브 트리 링크 필드
} treeNode;
```

❖ 트리 정렬 예제 프로그램

```
// 노드 x를 삽입하는 연산 : [예제 7-3] 26~41행
treeNode* insertNode(treeNode *p, element x) {
    treeNode *newNode;
    if (p == NULL) {
        newNode = (treeNode*)malloc(sizeof(treeNode));
        newNode->key = x;
        newNode->left = NULL;
        newNode->right = NULL;
        return newNode;
    }
    else if (x < p->key) p->left = insertNode(p->left, x);
    else if (x > p->key) p->right = insertNode(p->right, x);
    else printf("%n 이미 같은 키가 있습니다! %n");

    return p;
}
```

❖ 트리 정렬 예제 프로그램

```
// 이진 탐색 트리를 중위 순회하면서 경로를 출력하는 연산
void displayInorder(treeNode* root) {
    if (root) {
        displayInorder(root->left);
        printf("%d ", root->key);
        displayInorder(root->right);
    }
}

// 트리 정렬 연산
void treeSort(element a[], int n) {
    int i;
    treeNode* root = NULL; // 공백 이진 탐색 트리 생성
    root = insertNode(root, a[0]); // a[0]을 공백 이진 탐색 트리의 루트로 삽입

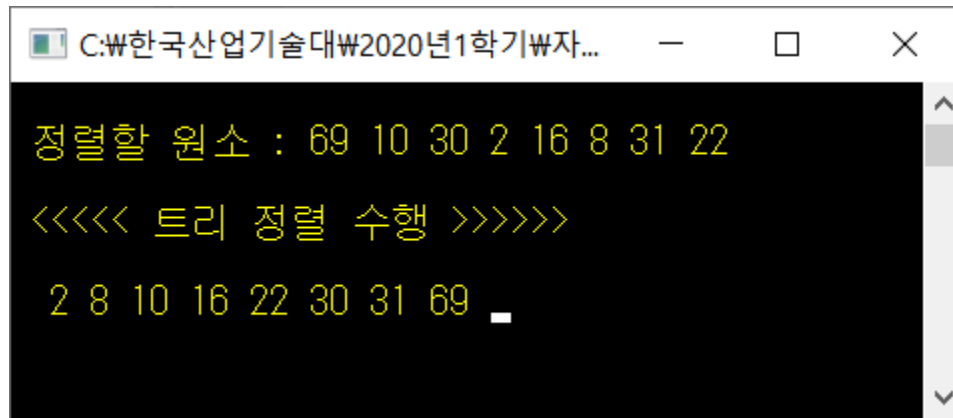
    for (i = 1; i < n; i++) // a[1]~a[n-1]의 원소들을
        insertNode(root, a[i]); // 삽입하면서 이진 탐색 트리 구성
    displayInorder(root); // 이진 탐색 트리를 중위 순회한 경로 출력
}
```

❖ 트리 정렬 예제 프로그램

```
int main() {  
    element list[8] = { 69, 10, 30, 2, 16, 8, 31, 22 };  
    int size = 8;  
    printf("\n <<<<< 트리 정렬 수행 >>>>> \n\n ");  
    treeSort(list, size);  
  
    getchar();  
}
```

❖ 트리 정렬하기 프로그램 : [교재 544p](#)

■ 실행 결과



```
C:\₩한국산업기술대₩2020년1학기₩자...  
정렬할 원소 : 69 10 30 2 16 8 31 22  
<<<< 트리 정렬 수행 >>>>>  
2 8 10 16 22 30 31 69 _
```